

P2 Streamer Programming Guide

Comprehensive Reference for Propeller 2 Streamer Hardware

November 2026

Version 1.1.4

Guide Organization

High-Speed I/O, Video, and Signal Processing with the P2 Streamer

Part I: Streamer Fundamentals

- Understanding the Streamer
- Architecture
- NCO and Timing
- Command Structure

Part II: Mode Reference

- Immediate Modes
- RDFAST Modes
- RGB Video Modes
- WRFAST Input Modes
- ADC Sampling Modes
- DDS/Goertzel Mode

Part III: Configuration Reference

- DAC Channel Configuration
- Pin Selection and Control
- Programming Constants
- Events and Synchronization

Part IV: Applications

- Video Output (VGA, HDMI, Composite)
- High-Speed Serial (SPI)
- Signal Processing
- Integration Patterns

Part V: Appendices

- Complete Mode Encoding Table
- Symbol Quick Reference
- Frequency Calculation Tables
- Troubleshooting Guide
- Index

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Contents

Copyright and License **7**
Acknowledgments . . . 7
Sources . . . 8
How to Use This Guide . . . 8
Document Conventions . . . 8
Enhancement Markers . . . 8
Code Blocks . . . 9

Part I: Streamer Fundamentals **10**

Chapter 1: Understanding the Streamer **10**
1.1 What the streamer is . . . 10
1.2 Why the streamer exists . . . 11
1.3 A mental model: a paced, configurable pipe . . . 11
1.4 Two directions: output and input . . . 12
1.5 Why there are so many modes — and why they differ . . . 12
1.6 The special capabilities, in plain words . . . 12
1.7 If you're building... . . . 13
1.8 What each cog actually has . . . 13

Chapter 2: Architecture **15**
2.1 Block Diagram . . . 15
2.2 Data Flow Paths . . . 16
2.3 Component Functions . . . 17

Chapter 3: NCO and Timing **18**
3.1 NCO Operation . . . 18
3.2 Frequency Calculation . . . 19
3.3 Setting NCO Frequency . . . 19
3.4 Choosing a Pixel Rate . . . 20
3.5 Clock Accuracy and Jitter . . . 21

Chapter 4: Command Structure **23**
4.1 Command Word Format . . . 23
4.2 Mode Field D[31:28] . . . 23

4.3 DAC Routing Field D[27:24] . . . 24
4.4 Enable/Write Field D[23] . . . 24
4.5 Pin Group Field D[22:20] . . . 24
4.6 Count Field D[15:0] . . . 24
4.7 Streamer Instructions . . . 25

Part II: Mode Reference **26**

Chapter 5: Immediate Modes **26**
5.1 Immediate → LUT → Pins/DACs . . . 27
5.2 Immediate → Pins/DACs . . . 27

Chapter 6: RDFAST Modes **29**
6.1 RDFAST → LUT → Pins/DACs . . . 29
6.2 RDFAST → Pins/DACs . . . 30

Chapter 7: RGB Video Modes **32**
7.1 RGB Format Modes . . . 32
7.2 Color Format Details . . . 33
7.3 RGB Mode Example . . . 34

Chapter 8: WRFAST Input Modes **35**
8.0 What the Capture Path Actually Reads . . . 35
8.1 Pin Capture Modes . . . 37

Chapter 9: ADC Sampling Modes **38**
9.1 ADC Capture Modes . . . 38
9.2 ADC Configuration Example . . . 38

Chapter 10: DDS/Goertzel Mode **40**
10.1 Mode Variants . . . 40
10.2 Operation . . . 41
10.3 The LUT Window . . . 41
10.4 LUT Setup . . . 42
10.5 SINC1 vs SINC2 . . . 43
10.6 Reading Results . . . 44
10.7 Frequency Calculation . . . 44

Part III: Configuration Reference **45**

Chapter 11: DAC Channel Configuration **45**
11.0 Getting a DAC Channel Onto a Pin . . . 45
11.1 DAC Routing Table . . . 47
11.2 DAC Pin Mapping . . . 48

11.3 Common DAC Configurations 48

Chapter 12: Pin Selection and Control **50**
12.0 Reading a Pin Field 50
12.1 Pin Group Selection 51
12.2 Sub-Pin Selection 52
12.3 Enable Control 53
12.4 Alternate Bit Order 53

Chapter 13: Programming Constants **54**
13.1 Mode Symbols 54
13.2 Control Symbols 55
13.3 DAC Symbols 56
13.4 Symbol Composition 57

Chapter 14: Events and Synchronization **59**
14.1 Streamer Events 59
14.2 Event Instructions 59
14.3 Event Clearing 60
14.4 Synchronization Patterns 60
14.5 Debugging Streamer Code 61

Part IV: Applications **62**

Chapter 15: Video Output **62**
15.0 The Colorspace Converter 62
15.1 VGA Output 66
15.2 HDMI/DVI Output 68
15.3 Composite Video 72

Chapter 16: High-Speed Serial (SPI) **73**
16.1 SPI Output with Streamer 73
16.2 Coordinating with WAITXFI 74

Chapter 17: Signal Processing **75**
17.1 Goertzel Frequency Detection 75
17.2 DDS Waveform Generation 77

Chapter 18: Integration Patterns **80**
18.1 Double Buffering 80
18.2 Multi-Cog Video 80
18.3 Streamer + Smart Pin Coordination 81

Part V: Appendices **82**

Appendix A: Complete Mode Encoding Table **82**

Appendix B: Symbol Quick Reference **85**

Mode Symbols 85

Control Symbols 85

DAC Symbols 85

Appendix C: Frequency Calculation Tables **86**

NCO Frequency Values 86

Common Video Pixel Rates 86

Appendix D: Troubleshooting Guide **87**

Symptom: No Output on Pins 87

Symptom: Corrupted Data from RDFAST 87

Symptom: Streamer Stops Unexpectedly 87

Symptom: Phase Drift in Video 87

Symptom: DAC Output Incorrect 87

Symptom: Goertzel Results Invalid 88

Symptom: Measurements Change When You Add DEBUG 88

Index **89**

List of Figures

2.1 Streamer output path: hub to pins, through the cog's hub FIFO set up by RDFAST . 15
2.2 Streamer input paths: pins to hub, through the same hub FIFO set up by WRFAST; and the Goertzel analyzer 15
2.3 Streamer data-flow paths 16

3.1 NCO phase accumulation and rollover 18

4.1 Command word: D operand of XINIT / XCONT / XZERO 23

7.1 Source-format bit packing. LUMA8 and RGBI8 both select one of eight colors and carry a luminance value – LUMA8 takes its color from S[2:0], RGBI8 from the pixel's top three bits. Only RGB8/16/24 carry per-channel values. 33

10.1 DDS output with Goertzel accumulation (every clock). The window is 4 to 512 entries, selected by S[11:9]; %A bounds it and %T offsets playback within it. 41

15.1 VGA 640×480 horizontal line timing 66

Copyright and License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

To view the full license, visit: https://creativecommons.org/licenses/by-sa/4.0/

Trademarks

Parallax, Propeller, Spin, and the Parallax logo are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Acknowledgments

This guide would not exist without the contributions of many individuals and organizations:

Parallax Inc. for creating the Propeller 2 microcontroller and providing the comprehensive reference documentation that forms the foundation of this work.

Chip Gracey for the design of the P2 streamer, NCO, and colorspace converter, and for maintaining the detailed silicon documentation that defines their behavior.

The P2 Community for the video, audio, and signal-processing drivers whose real-world usage informed the examples and patterns in this guide.

Sources

This guide draws on the following primary and community sources:

- **Parallax Propeller 2 Documentation v35 - Rev B/C** (Chip Gracey, Parallax Inc.) — streamer architecture, mode encodings, NCO and DDS/Goertzel behavior
- **Spin2 Reference Manual v51** (Parallax Inc.) — built-in streamer symbols and language integration
- **Community video and Goertzel drivers** (Parallax OBEX) — application patterns

How to Use This Guide

This reference serves developers implementing high-speed I/O on the Propeller 2. It assumes familiarity with P2 cog/hub architecture, basic PASM2 instructions, and RDFAST/WRFAST FIFO operations.

Structure:

- **Part I (Fundamentals)** establishes the mental model — read this first to understand how the streamer operates
- **Part II (Mode Reference)** documents all streamer modes — use for specific mode lookup
- **Part III (Configuration)** covers cross-cutting concerns — DAC routing, pin selection, symbols
- **Part IV (Applications)** provides implementation patterns — video, SPI, signal processing
- **Appendices** contain quick-reference tables and troubleshooting guidance

Document Conventions

Element	Format	Example
Instructions	Bold uppercase	**XINIT**, **XCONT**
Symbols	Monospace	`X_RFWORD_RGB16`
Bit fields	Brackets	D[31:28], S[19:16]
Binary	Percent + underscores	`%1011_0000`
Hexadecimal	Dollar prefix	`$B085_0000`

Enhancement Markers

Three colored callout boxes set “things to know” apart from the running text:

- **CAUTION** (amber) — common mistakes with non-obvious consequences
- **TIP** (teal) — non-obvious techniques or optimizations
- **HARDWARE** (graphite) — silicon-level details affecting usage

Code Blocks

Every PASM2 and Spin2 block in this guide is one of three things, and you can tell which without running it:

- **No label** — a worked example, and the absence of a label is the promise. Every symbol it uses is defined in the block or cross-referenced to the section that defines it; the storage it writes is declared; and where it drives pins or DACs, the configuration that makes them emit is shown or cross-referenced. Nothing is left for you to fill in.
- **A bolded Pattern or Fragment lead-in on the line above** — a shape rather than a program. The lead-in names exactly what you supply.
- **ANTIPATTERN** (red) — code shown in order to be rejected, placed beside the form that works.

Part I: Streamer Fundamentals

The streamer rewards a little grounding before the encodings. This Part builds the mental model — what the streamer is, how its NCO sets the pace, and the command vocabulary you will use everywhere else — so the mode tables in Part II read as deliberate choices rather than magic.

Chapter 1: Understanding the Streamer

Before the mode tables and bit fields, it helps to know what the streamer *is*, why the P2 has one, and how to think about it when you sit down to build something. This chapter builds that picture. The rest of the guide is the detailed reference; this chapter is the map.

1.1 What the streamer is

Every cog on the P2 has its own **streamer**: a small, tireless engine that moves data between hub memory and the outside world — the pins, the DAC channels, the ADC inputs — entirely on its own, at a rate you choose. Once you start it, it runs without the cog's help. Your code can compute, make decisions, or sleep while the streamer keeps feeding pixels to a display or pulling samples off a wire.

The detail that makes the streamer special is that it carries **its own clock — and you set its rate**. A piece of hardware called the NCO (Numerically-Controlled Oscillator) is the streamer's adjustable metronome: it ticks at whatever rate your application needs, and the streamer moves one piece of data on each tick. You dial that rate in directly — a ~25 MHz pixel rate for VGA, a 48 kHz sample rate for audio, or anything else — and its average rate holds to better than a part in ten million. (How evenly the individual ticks fall is a separate matter, and §3.4 shows how to make them identical.) That precise, self-kept timing is what lets a single cog produce a clean video picture or an unwavering audio stream without ever having to babysit the timing in software.

And because the streamer lives *inside* the cog, **each cog has its own streamer and its own NCO** — so the eight streamers are clocked independently. They do not share a rate. One cog can push video pixels at 25 MHz while another streams audio at 48 kHz and a third samples an ADC at some other rate entirely, all at the same moment, each running at exactly the rate its job requires.

> **If you've used DMA before:** the streamer is a close cousin of a DMA channel, with two important additions. First, it has that built-in metronome, so it does *paced* transfers at an exact sample rate rather than "as fast as the bus allows." Second, it reshapes data

as it moves — packing bits, expanding through a palette, converting color formats — instead of copying bytes verbatim. If you have never met DMA, don't worry: everything below stands on its own.

1.2 Why the streamer exists

Generating precise, fast, repetitive signals in software is brutally expensive. Imagine driving a video display by hand: your code would have to write a new color to the pins every forty nanoseconds, forever, without ever slipping. A single loop like that would consume an entire cog and still stutter the moment anything interrupted it. The streamer exists so that this relentless, timed, repetitive work happens in hardware, leaving the cog free for the *interesting* part — drawing the next frame, decoding the next packet, running the game.

So why is it so complicated? Because one engine has to serve very different jobs: pushing video to a screen, playing audio through a DAC, capturing pins like a logic analyzer, sampling an analog input like an oscilloscope, generating tones, even detecting a specific frequency in an incoming signal. Rather than give each cog six narrow peripherals, the P2 gives it **one highly configurable engine**. The configurability is the complexity — and it is also the payoff. Learn the handful of knobs once, and the same engine does all of those jobs.

1.3 A mental model: a paced, configurable pipe

Picture a pipe running from hub memory to the pins, with an adjustable shaper in the middle and a metronome setting the pace. Every streamer command you issue is just a way of answering four questions about that pipe:

1. **Where does the data come from?** An immediate value you supply, or a stream pulled from hub memory through the FIFO.
2. **Where does it go?** Out to the pins, out to the DAC channels — or, when capturing, back into hub memory.
3. **How is it shaped along the way?** Sliced into 1-, 2-, 4-, 8-, 16-, or 32-bit pieces; expanded through a lookup table (a palette); or converted into a video color format.
4. **How fast?** The NCO's beat.

That is the whole idea. The long list of modes in Part II is nothing more than the *useful combinations* of those four answers. Once you read a mode as "hub data, to the pins, one byte at a time, at the pixel rate," the names stop looking like alphabet soup.

1.4 Two directions: output and input

The streamer works in two directions, and they are different enough that it helps to think about them separately.

Output modes drive the world — pixels to a screen, samples to a speaker, bits down a serial line. Here the interesting questions are *where the data comes from* (a buffer in memory, or a small repeating table) and *how it is shaped* (raw bytes, palette lookup, RGB color) before it reaches the pins.

Input modes capture the world — pin states recorded into memory (a logic analyzer); ADC readings recorded into memory (an oscilloscope). Here the interesting question is *how the captured data is packed* as it is written back to hub.

Knowing which direction you are working in immediately cuts the mode list roughly in half. The split is a guide rather than a rule: DDS/Goertzel generates a waveform on the DAC channels while it measures an input (Chapter 10).

1.5 Why there are so many modes — and why they differ

The mode tables are not a random pile of similar-looking options: they vary along just a few axes, and each combination earns its place by doing a *distinct* job. Two quick contrasts make the point:

- **Playing a recording vs. synthesizing a tone.** Both end at the DAC. But playing a recorded sound streams a long buffer out of hub memory, while synthesizing a tone loops a *tiny* table — one cycle of a sine wave — over and over, with the NCO setting the pitch. Same destination, completely different source strategy. That difference is exactly what separates the two modes.
- **One channel vs. multichannel audio.** These are not different engines at all — they are one routing knob (the DAC-routing field) deciding how many of the four DAC channels get fed. "Stereo" is a setting, not a separate mode.

So when two mode names sound alike, the question to ask is: *which of the four pipe-questions do they answer differently?* That is always where the real distinction lives.

1.6 The special capabilities, in plain words

A few streamer features have intimidating names. Here is what they actually mean.

Video (RGB and colorspace conversion). The streamer can pull pixels from a framebuffer in memory and push them out as an analog VGA signal or a digital HDMI signal, translating color formats as it goes. You provide the picture; the streamer handles the relentless pixel timing. *(Chapters 7 and 15.)*

DDS — Direct Digital Synthesis. A grand name for a simple trick: generate a repeating waveform — a sine, a tone, any shape you like — by stepping through a small table at a precise rate. The NCO sets the frequency, so the same table produces any pitch. This is how you build a function generator, an audio tone, or a modulated carrier. *(Chapters 10 and 17.)*

Goertzel frequency analysis. This one is never obvious from its name, so plainly: it is a way to ask, in hardware, *"how much of one specific frequency is present in this incoming signal?"* You tell it the frequency you care about, and it reports how strongly that tone is there. The textbook use is decoding telephone touch-tones — the beeps of a phone keypad, known as DTMF — but the same trick detects whistles and alarm tones, measures distance with ultrasound (send a ping, listen for its echo), and builds simple receivers. It is cheap precisely because it checks only the *one* frequency you ask about, instead of computing a whole spectrum. *(Chapters 10 and 17.)*

1.7 If you're building...

You usually arrive at the streamer with an application already in mind. Find it below; the chapters on the right are the ones worth reading closely first. The rest of the reference is there for when you need the exact bits.

If you're thinking about...	The streamer gives you...	Start at
Video (VGA, HDMI, composite)	color pixels from a framebuffer	Ch. 7, Ch. 15
Audio / sound output	DAC samples from a buffer	Ch. 5–6, Ch. 11
Generating tones or waveforms	DDS from a small table	Ch. 10, Ch. 17
A logic analyzer (capturing pins)	pin states written to memory	Ch. 8
Sampling an analog signal	ADC readings written to memory	Ch. 9
Detecting a specific tone or frequency	Goertzel analysis	Ch. 10, Ch. 17
High-speed serial (fast SPI)	timed bit output, clocked by a smart pin	Ch. 16

1.8 What each cog actually has

For all that capability, the hardware budget is modest. Each cog contains exactly one streamer, with:

- one 32-bit NCO (the metronome / phase accumulator);
- one command buffer (it holds one queued command, so commands can hand off without a gap);
- four 8-bit DAC channels (X0, X1, X2, X3);
- one Goertzel analyzer;
- access to the cog's LUT RAM (used as a palette or a waveform table).

And it leans on a few neighboring P2 subsystems:

Subsystem	What it provides
Hub FIFO	the data source / sink, via RDFAST / WRFAST
LUT RAM	palette lookups, sine/cosine tables
DAC channels	analog output for video and audio
Colorspace converter	composite video, and color matrixing on the DAC channels
Smart pins	clock generation and timing

With that picture in place, Chapter 2 opens up the engine itself — the data paths inside the streamer and how the pieces connect.

Chapter 2: Architecture

Chapter 1 described the streamer as a paced pipe from memory to the pins. This chapter opens that pipe up — the pieces inside, how data flows through them, and how the NCO drives the whole thing. You do not need this depth to *use* the streamer, but it makes the mode choices in Part II feel inevitable rather than arbitrary.

2.1 Block Diagram

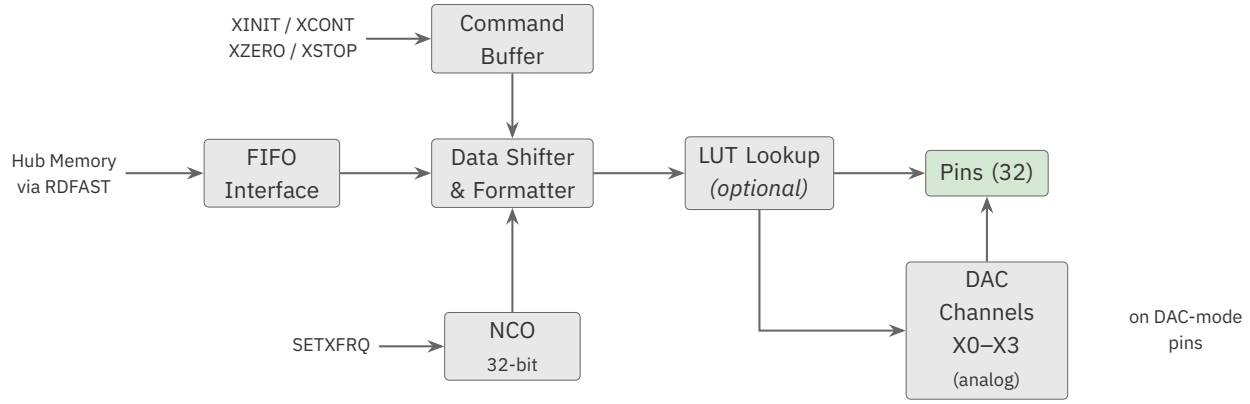


Figure 2.1. Streamer output path: hub to pins, through the cog's hub FIFO set up by RDFAST

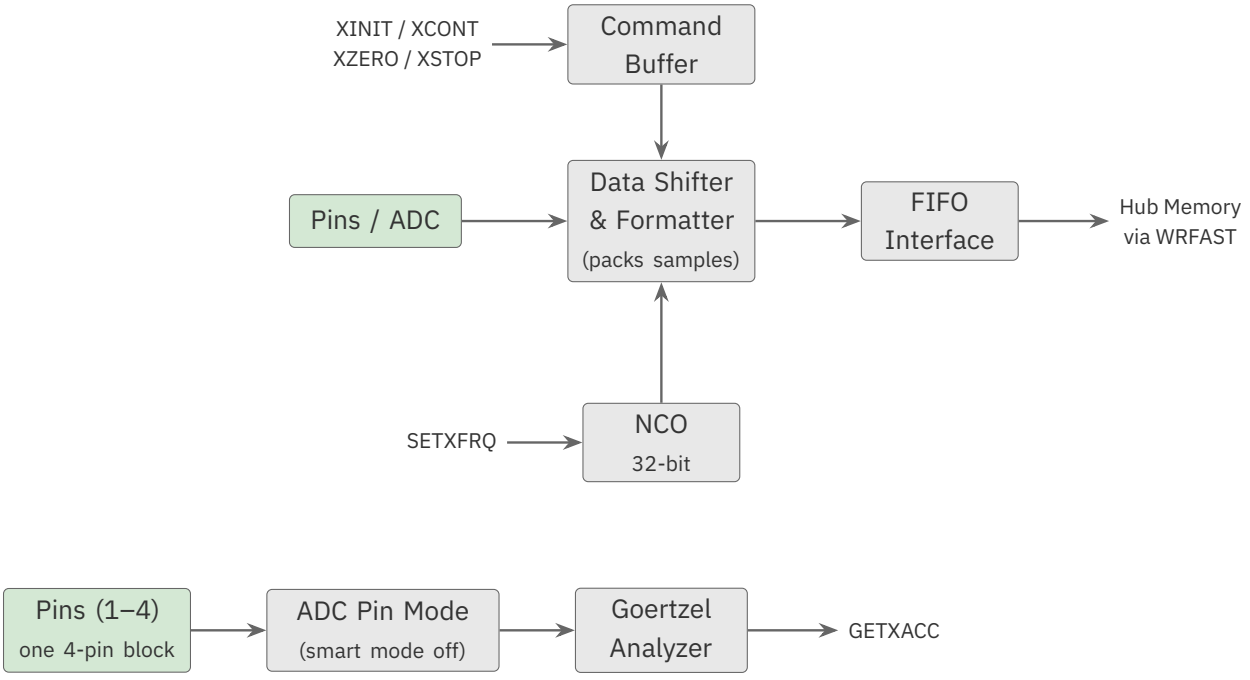


Figure 2.2. Streamer input paths: pins to hub, through the same hub FIFO set up by WRFAST; and the Goertzel analyzer

What the diagrams do not show is *which physical pins* DAC0–DAC3 — and the 32 output pins — actually land on, and you will care about that the moment you wire something up. The mapping is not arbitrary: each DAC channel can only drive pins whose number ends in its own two bits — DAC0 drives pins ending in `%00` (pins 0, 4, 8, …), DAC1 those ending in `%01`, and so on. The complete channel-to-pin mapping is in Chapter 11, and choosing which 32-pin group a command targets is Chapter 12. (Setting a pin up to *act* as an analog/DAC output is a pin-configuration topic in its own right — see the *P2 I/O & Smart Pins User Guide*.) For now, just note that the output diagram's "DAC Channels X0–X3" and "Pins" become specific pin numbers once you choose them.

2.2 Data Flow Paths

The streamer supports multiple data flow configurations:

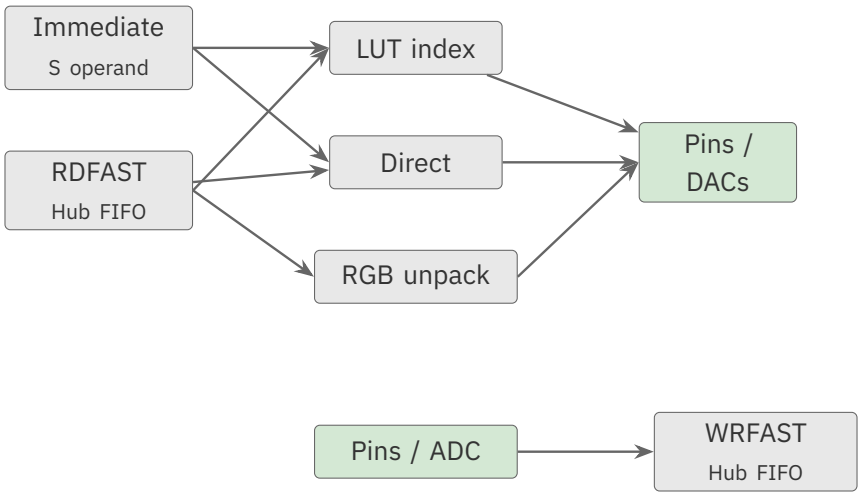


Figure 2.3. Streamer data-flow paths

Output Paths (hub → Pins/DACs):

1. **Immediate → LUT → Pins/DACs**: S operand indexes LUT; LUT data drives output
2. **Immediate → Pins/DACs**: S operand drives output directly
3. **RDFAST → LUT → Pins/DACs**: Hub data indexes LUT; LUT data drives output
4. **RDFAST → Pins/DACs**: Hub data drives output directly
5. **RDFAST → RGB → Pins/DACs**: Hub data is unpacked into red, green and blue

Input Paths (Pins → hub):

1. **Pins → DACs/WRFAST**: Pin states written to hub
2. **ADC → DACs/WRFAST**: ADC readings written to hub

Special Path:

1. **DDS/Goertzel**: LUT-based synthesis with simultaneous frequency analysis

2.3 Component Functions

NCO (Numerically-Controlled Oscillator): Times all streamer operations. On each clock, adds frequency value to phase accumulator. Rollover (MSB set) triggers data advancement.

Command Buffer: Holds one pending command. Enables seamless transitions between streamer operations without gaps.

Data Shifter: Handles data widths from 1-bit to 32-bit. Extracts and formats data according to mode.

LUT Interface: Reads cog LUT RAM for palette expansion or waveform generation. 512 entries × 32 bits.

DAC Channels: Four 8-bit channels (X0-X3) map to pins based on pin number LSBs. Configurable routing allows stereo, differential, or independent operation.

Goertzel Analyzer: Hardware frequency detection using Goertzel algorithm. Accumulates sine and cosine products for magnitude/phase extraction.

Chapter 3: NCO and Timing

The NCO is the streamer's metronome, and setting its rate correctly is the difference between a steady picture and a rolling one. This chapter shows how the NCO produces that beat and how to compute the value you need for a given rate.

3.1 NCO Operation

The NCO operates on every system clock:

```
phase = (phase & $7FFF_FFFF) + frequency
```

1. The MSB is masked (cleared) before addition
2. The frequency value adds to the phase accumulator
3. If the new MSB is set, a "rollover" occurs
4. On rollover, the streamer advances to the next data element

DDS/Goertzel does not wait for rollover. That mode advances on every system clock, using the NCO's phase as a running index into the LUT rather than as a step trigger (Chapter 10). Step 4 describes every other mode.

HARDWARE

The phase accumulator is a 32-bit register. Its most-significant bit is masked off before each addition and used as the rollover flag, so 31 bits accumulate the phase. Frequency resolution is therefore `clock_frequency / 2^31`.

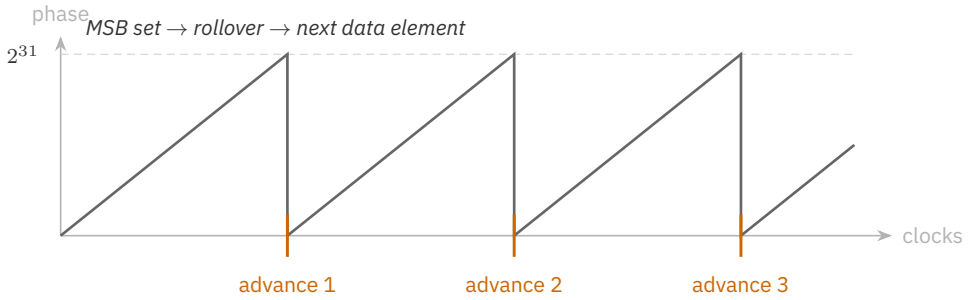


Figure 3.1. NCO phase accumulation and rollover

3.2 Frequency Calculation

Formula:

```
frequency = $8000_0000 × (desired_rate / clock_frequency)
```

Common Values:

Rate Ratio	Frequency Word	Exact?
1:1	$8000_0000	exact
1:2	$4000_0000	exact
1:4	$2000_0000	exact
1:8	$1000_0000	exact
1:3	$2AAA_AAAB	rounded (+1)
1:10	$0CCC_CCCD	rounded (+1)

A **power-of-two ratio** divides `$8000_0000` evenly, so its word is exact; every other ratio must be rounded up (the +1 convention below). A power-of-two ratio also makes the sysclk an exact integer multiple of the pixel rate, and that integer ratio — not the word's exactness — is what removes per-pixel jitter (see §3.4 for choosing a rate around it). Appendix C lists the full set of ratio and pixel-rate values.

CAUTION

Round up, and never let the value reach zero. Truncating `$8000_0000 * rate/clock` leaves the frequency word a hair short of a clean rollover, so the streamer's *first* rollover lands one clock late and the timing is skewed from there. Round the result up instead — or add 1 to a truncated value. This is the **+1 convention** from the *Parallax Propeller 2 Documentation v35 - Rev B/C*: its HDMI example sets the 1/10 rate as `$0CCC_CCCC + 1`, because *"the +1 forces initial NCO rollover on the 10th clock."* The same habit guards against a second, nastier failure — a frequency word of **zero never rolls over at all, so the streamer stalls forever**. When a calculation could land low (or on zero), round up. The common-values table above already includes the +1 where the exact ratios need it.

3.3 Setting NCO Frequency

Method 1: SETXFRQ instruction

```
        ' 1/10 clock = 25 MHz; the +1 forces the first rollover
        SETXFRQ #$0CCC_CCCC+1
```

The `+1` is the rounding-up habit from the pitfall above: `$0CCC_CCCC` is the truncated 1/10 value, and adding 1 makes the streamer roll over on the 10th clock instead of the 11th (and keeps the word off zero). You will see this `+1` throughout the examples.

Method 2: SETQ before streamer command. Pattern — supply `mode` and `data`.

```
        SETQ    ##$0CCC_CCCC+1          ' Frequency in Q
        XINIT   mode, data            ' Uses Q as frequency
```

The **SETQ** method allows changing frequency atomically with a new command.

3.4 Choosing a Pixel Rate

Two facts decide this, and the first is counter-intuitive:

1. **The average pixel clock is essentially exact at any sysclk.** The frequency word is 32-bit, but because the phase accumulator masks its MSB each clock, only 31 bits accumulate — so the resolution is `sysclk / 2^31` ≈ 0.12 Hz at 250 MHz. The error in the *average* output rate is under ~0.01 ppm — far below any monitor's tolerance. Frequency accuracy is **not** what you tune.
2. **Per-pixel jitter is what varies.** Each pixel lasts a whole number of sysclk cycles. When `sysclk / pixel_clock` is an integer, every pixel is identical — **no jitter**. When it is not, pixel widths swing by ±1 sysclk cycle around the ideal (the average still comes out exact). So the rule is: **pick a sysclk that is an integer multiple of the pixel clock.**

The jitter-free sysclks below are the integer multiples the P2 PLL can actually produce from a 20 MHz crystal. The last column is the penalty for ignoring the rule and just running 250 MHz.

Mode	Pixel clock	Jitter-free sysclks (× pixel clock)	At 250 MHz
VGA 640×480	25.175 MHz	none on a 20 MHz crystal — see note	use 25.0 MHz pixel → 10.000 cyc/px, no jitter
480p 720×480	27.0 MHz	270 (×10), 297 (×11), 324 (×12)	9.26 cyc/px → ~11% jitter
SVGA 800×600	40.0 MHz	160 / 200 / 240 / 280 / 320 (×4–×8)	6.25 cyc/px → ~16% jitter
XGA 1024×768	65.0 MHz	130 / 195 / 260 / 325 (×2–×5)	3.85 cyc/px → ~26% jitter
720p 1280×720	74.25 MHz	148.5 (×2), 297 (×4)	3.37 cyc/px → ~30% jitter

> **VGA note:** 25.175 MHz's exact multiples (201.4, 251.75 MHz) cannot be produced by the P2 PLL from a 20 MHz crystal. Standard practice is a **25.0 MHz pixel clock at 250**

MHz sysclk — exactly 10 cycles per pixel (jitter-free), with the clock 0.7% slow, which monitors absorb. The streamer's DVI output needs a sysclk of ten times the pixel rate (§15.2), which caps it near the low tens of MHz of pixel clock; 1080p would need a 1.485 GHz serial clock and is out of reach.

The **SETXFRQ** word for any combination is `$8000_0000 * pixel_clock / sysclk`, truncated, and then incremented by 1 if that division left a remainder — the lookup tables in Appendix C list common values. Worked both ways:

```
Example 1 — integer ratio: SVGA 800×600, 40 MHz pixel @ 320 MHz
  WORD     = $8000_0000 × 40 / 320 = $8000_0000/8 = $1000_0000
             divides exactly, so no increment
  achieved = 320 × $1000_0000 / $8000_0000 = 40.000 MHz   (exact)
  cyc/px   = 320 / 40 = 8.000   -> no jitter

Example 2 — non-integer ratio: VGA 640×480, 25.175 MHz @ 250 MHz
  WORD     = $8000_0000 × 25.175 / 250 = $0CE3_BCD3 remainder
             remainder is non-zero, so increment  -> $0CE3_BCD4
  achieved = 250 × $0CE3_BCD4 / $8000_0000 = 25.175 MHz   (<0.01 ppm)
  cyc/px   = 250 / 25.175 = 9.93   -> ±1-cycle jitter (~10% of pixel)
  remedy   = 25.0 MHz @ 250 = 10.000 cyc/px   -> no jitter
```

TIP

The achieved pixel clock is essentially exact at **any** sysclk — what you manage is per-pixel jitter, not frequency error. Make `sysclk / pixel_clock` a whole number and every pixel is the same width.

3.5 Clock Accuracy and Jitter

Accuracy and jitter are independent. The jitter in §3.4 comes from the sysclk-to-pixel *ratio* and is the same no matter how precise your oscillator is. **Absolute accuracy** — how close the pixel clock sits to its exact nominal frequency, and how steadily it holds — is set entirely by the reference crystal: the PLL only multiplies it (`sysclk = crystal * M / (D*P)`) and the NCO adds under 0.01 ppm, so your pixel clock is **no more accurate than your crystal**.

If you build on a Parallax **P2 Edge module**, this is already handled for you.

HARDWARE

The P2 Edge modules carry a 20 MHz TCXO rated ±0.5 ppm (temperature-compensated). Every clock the P2 derives is referenced to it, so your pixel clocks, sample rates, and color subcarriers come out accurate to ~0.5 ppm and hold that across temperature — with no effort on your part. The Parallax module guides specify it as a high-precision oscillator, which makes the Edge a safe default for accurate-timing work as well as general projects.

On an **externally designed board** the P2 runs from whatever crystal you fit, and the PLL can only multiply that reference — it cannot make the clock more accurate than its source. A general-purpose crystal is typically tens of ppm, with additional drift over temperature, and that error flows straight through to every streamer rate.

For most video this is a non-issue: monitors absorb thousands of ppm of pixel-clock error — the same tolerance that makes §3.4's 25.0-for-25.175 substitution invisible. It bites only when the *absolute* frequency is the deliverable: NTSC/PAL composite **colorburst** (a few ppm before the hue drifts), precise audio sample rates, or long-running timing. **So if you are designing a custom board for video — composite especially — choose the crystal with this in mind, or fit a TCXO; an Edge module gives you that precision out of the box.**

Chapter 4: Command Structure

A streamer command is a single value — the D operand — that packs together every choice from Chapter 1's four questions: what mode, where the data goes, which pins, and how long to run. This chapter lays that packed word out field by field, then introduces the small set of instructions (XINIT, XCONT, XZERO) that start and chain commands.

4.1 Command Word Format

The D operand to **XINIT**, **XCONT**, and **XZERO** contains:



Figure 4.1. Command word: D operand of XINIT / XCONT / XZERO

4.2 Mode Field D[31:28]

D[31:28]	Category	Data Source	Data Destination
%0000-%0011	IMM→LUT	S operand	LUT → Pins/DACs
%0100-%0111	IMM→Direct	S operand	Pins/DACs
%0111	RF→LUT	RDFAST	LUT → Pins/DACs
%1000-%1011	RF→Direct	RDFAST	Pins/DACs
%1011	RF→RGB	RDFAST	RGB unpack → Pins/DACs
%1100-%1111	Capture	Pins	DACs/WRFAST
%1111	ADC	ADC	DACs/WRFAST
%1111_x111	DDS/Goertzel	LUT	DACs + Analysis

> **Note:** Rows show the 4-bit **mode nibble** D[31:28]. Several nibbles serve more than one family and are separated by the config field D[19:16] — %0111, %1011 and %1111 each appear twice above. DDS/Goertzel is written %1111_p111 to show both fields at once; Appendix A gives every mode's full template. Separately — outside that config field — bit D[23] selects SINC1 (0) or SINC2 (1).

4.3 DAC Routing Field D[27:24]

The %dddd field controls DAC channel assignment. See Chapter 11 for the complete routing table.

4.4 Enable/Write Field D[23]

Mode Type	D[23] = 0	D[23] = 1
Output modes	Pins disabled	Pins enabled
Input modes	WRFAST disabled	WRFAST enabled

4.5 Pin Group Field D[22:20]

Selects which 32-pin block the streamer addresses:

%ppp	Pin Range
%000	Pins 31..0
%001	Pins 39..8
%010	Pins 47..16
%011	Pins 55..24
%100	Pins 63..32
%101	Pins 7..0, 63..40 (wrap)
%110	Pins 15..0, 63..48 (wrap)
%111	Pins 23..0, 63..56 (wrap)

4.6 Count Field D[15:0]

Specifies the number of NCO rollovers before the command completes.

- A count of 1 to 65,534 transfers that many data elements, then the command completes
- A count of $FFFF (65,535) streams **perpetually** — the command runs until a new command is issued or **XSTOP** stops it
- A count of 0 stops the streamer (this is exactly what **XSTOP** / XINIT #0,#0 does)

4.7 Streamer Instructions

Instruction	Syntax	Effect
SETXFRQ	`SETXFRQ {#}D`	Set NCO frequency
XINIT	`XINIT {#}D,{#}S`	Start immediately, zero phase
XCONT	`XCONT {#}D,{#}S`	Buffer command, continue phase
XZERO	`XZERO {#}D,{#}S`	Buffer command, zero phase
XSTOP	`XSTOP`	Stop immediately (alias: `XINIT #0,#0`)
GETXACC	`GETXACC D`	Get Goertzel accumulators

XINIT starts the streamer immediately, interrupting any current operation and zeroing the NCO phase.

XCONT and **XZERO** buffer a command that executes on the final rollover of the current command. **XCONT** preserves NCO phase; **XZERO** resets it.

TIP

Use **XZERO** at video line boundaries to prevent phase drift accumulation across lines.

CAUTION

XCONT and XZERO are for seamless command-to-command continuity, not for starting the streamer. They wait for the current command's final NCO rollover; if the streamer is already idle (count = 0) there is no wait and the command runs immediately — and XCONT begins with whatever phase remains in the accumulator rather than a known zero. Use **XINIT** to start the streamer from a clean, phase-zeroed state.

A perpetual command is the third case. A command issued with the maximal count `$FFFF` (§4.6) runs without decrementing its counter, so it never reaches a final rollover to wait for. A buffered XZERO/XCONT behind one waits only for the **next** NCO rollover and then takes over — one rollover, not the remainder of a transfer.

Part II: Mode Reference

The streamer's modes are the heart of this reference. This Part documents each family in turn — immediate, hub-streamed, video, pin-capture, ADC, and the special DDS/Goertzel mode. Each chapter opens with what its modes are *for* before giving the exact encodings.

Chapter 5: Immediate Modes

Immediate modes are the simplest place to start. Instead of streaming from memory, the data you want to output is a value you hand the streamer directly, in the S operand. Reach for them when you have a small, fixed pattern to emit — a handful of pixels, a test pattern, a short bit sequence — and do not want to set up a hub buffer. The data can go straight to the pins and DACs, or pass through the LUT for palette expansion.

HARDWARE

When the count outruns the packed values, the last value repeats. An immediate mode's S operand holds a fixed number of sub-values — 32 one-bit, 4 eight-bit, 1 thirty-two-bit — but `D[15:0]` can ask for any count. Past the last packed value, that value is re-emitted on every remaining rollover rather than wrapping back to the first. That is what lets a single immediate value hold a steady level for an arbitrary interval: §15.1's blanking and sync intervals use `X_IMM_1X32_4DAC8`, whose S operand is one 32-bit value, and stream it for up to 800 pixels.

The RDFAST families of Chapter 6 behave differently — there the last value triggers the next hub fetch, so they never run out of data to send.

5.1 Immediate → LUT → Pins/DACs

The S operand provides index values into the LUT. LUT data drives pins and DACs.

The pin half needs no pin setup. The DAC half does: a DAC channel these modes feed only becomes a voltage on a pin configured per §11.0.

Mode	Symbol	Elements	Bits/Element
%0000	X_IMM_32X1_LUT	32	1
%0001	X_IMM_16X2_LUT	16	2
%0010	X_IMM_8X4_LUT	8	4
%0011	X_IMM_4X8_LUT	4	8

D[19:16] Field: LUT base address bits [8:5] (`%bbbb` → LUT address `%bbbb_00000`)

S Operand: 32-bit immediate data containing packed index values

Operation: On each NCO rollover, the next index value selects a LUT entry. The LUT long drives all 32 pins and/or DAC channels.

Example. Pattern — supply the LUT palette; §10.4 shows a table being built and bulk-loaded.

```
' Output 32 1-bit pixels using 2-entry palette at LUT $000
        DRVL    ##31<<6                  ' enable P31..P0 (11.0)
        XINIT   ##X_IMM_32X1_LUT | X_PINS_ON + 32, ##$AAAA_5555
```

5.2 Immediate → Pins/DACs

The S operand drives pins and DACs directly without LUT lookup. The DAC-channel columns below reach a pin only through the full setup in §11.0. The pin columns need none of that mode configuration — but they still need the pin **enabled**, because the streamer supplies a pin's output *state*, never its output *enable*. §11.0 draws the line.

Mode	Symbol	Pins	DAC Channels	DAC Bits
%0100	X_IMM_32X1_1DAC1	1	1	1
%0101	X_IMM_16X2_2DAC1	2	2	1
%0101	X_IMM_16X2_1DAC2	2	1	2
%0110	X_IMM_8X4_4DAC1	4	4	1
%0110	X_IMM_8X4_2DAC2	4	2	2
%0110	X_IMM_8X4_1DAC4	4	1	4
%0110	X_IMM_4X8_4DAC2	8	4	2
%0110	X_IMM_4X8_2DAC4	8	2	4
%0110	X_IMM_4X8_1DAC8	8	1	8
%0110	X_IMM_2X16_4DAC4	16	4	4
%0111	X_IMM_2X16_2DAC8	16	2	8
%0111	X_IMM_1X32_4DAC8	32	4	8

D[19:16] Field: Mode variant selector

S Operand: Packed data values

Example. Pattern — supply `pin`, a multiple of 8 for an 8-pin transfer (§12.0).

```
' Output 4 bytes to an 8-pin group, 8 bits each
        DRVL    #7<<6 + pin               ' enable those 8 pins (11.0)
        XINIT   ##X_IMM_4X8_1DAC8 | X_PINS_ON | (pin<<17) | 4, ##$12345678
```

Chapter 6: RDFAST Modes

RDFAST modes are the workhorse of the streamer. Where immediate modes carry a single fixed value, these stream a continuous flow of data out of hub memory — a framebuffer, an audio clip, a bitmap — onto the pins or DACs. This is what you use for anything longer than a few elements. The data arrives through the FIFO, which must be primed with **RDFAST** before the streamer command runs.

CAUTION

Run RDFAST before any RDFAST streamer command. It primes the cog's hub FIFO to *deliver* data (hub → streamer); until it does, the FIFO is not pointed at your buffer and the streamer pulls undefined data. The same FIFO handles the opposite direction via WRFAST (Chapter 8), so a cog streams one way at a time.

6.1 RDFAST → LUT → Pins/DACs

Hub data serves as LUT index values. As in Chapter 5, the DAC side of these modes needs the pin setup of §11.0 and the pin side does not.

> **Reading the `%MMMM_cccc` shorthand:** the two underscored nibbles are the **mode** field D[31:28] and the **config** field D[19:16] — *not* a single contiguous byte. D[27:20] sit between them in the command word (they carry DAC routing, enable, and pin-group fields; see §4.2). The shorthand pairs the two nibbles that pick the mode so a row reads at a glance; Appendix A lists every field in its own column.

Mode	Symbol	Hub Read	Elements	Bits/Element
`%0111_001a`	`X_RFLONG_32X1_LUT`	RFLONG	32	1
`%0111_010a`	`X_RFLONG_16X2_LUT`	RFLONG	16	2
`%0111_011a`	`X_RFLONG_8X4_LUT`	RFLONG	8	4
`%0111_1000`	`X_RFLONG_4X8_LUT`	RFLONG	4	8

S[3:0]: LUT base address bits [8:5]

%a bit: bit order *within each byte* — 0 = bottom-first, 1 = top-first. It reorders bits inside a sub-byte group, so it acts only on the 1-, 2- and 4-bit rows above; the 4×8 row has no `%a` bit and nothing for it to reorder.

CAUTION

Hub symbols in a Spin2 object. The hub-symbol patterns in this guide are written for a PASM-only file, where hub addresses are absolute. In a Spin2 object's DAT block, hub addresses are relative to the object: `##hubsymbol` assembles to $400 plus an offset, and `##@hubsymbol` to the offset from the start of the object, so the streamer would read other memory. In a Spin2 object, pass hub addresses in from Spin2 with `@`, through PTRA.

Example. Pattern — supply `bitmap_addr`, `base` — a multiple of 8 (§12.0) — and the LUT palette.

```
' Setup FIFO
        RDFAST  #0, ##bitmap_addr

' Stream 640 pixels through 256-color palette at LUT $000
        DRVL    ##31<<6 + base            ' enable the 32-pin window (11.0)
        XINIT   ##X_RFLONG_4X8_LUT | X_PINS_ON | (base<<17) | 640, #0
```

6.2 RDFAST → Pins/DACs

Hub data drives pins and DACs directly. The DAC-channel columns below reach a pin only through the setup in §11.0.

Mode	Symbol	Hub Read	Pins	DAC Channels	DAC Bits
%1000	X_RFBYTE_1P_1DAC1	RFBYTE	1	1	1
%1001	X_RFBYTE_2P_2DAC1	RFBYTE	2	2	1
%1001	X_RFBYTE_2P_1DAC2	RFBYTE	2	1	2
%1010	X_RFBYTE_4P_4DAC1	RFBYTE	4	4	1
%1010	X_RFBYTE_4P_2DAC2	RFBYTE	4	2	2
%1010	X_RFBYTE_4P_1DAC4	RFBYTE	4	1	4
%1010	X_RFBYTE_8P_4DAC2	RFBYTE	8	4	2
%1010	X_RFBYTE_8P_2DAC4	RFBYTE	8	2	4
%1010	X_RFBYTE_8P_1DAC8	RFBYTE	8	1	8
%1010	X_RFWORD_16P_4DAC4	RFWORD	16	4	4
%1011	X_RFWORD_16P_2DAC8	RFWORD	16	2	8
%1011	X_RFLONG_32P_4DAC8	RFLONG	32	4	8

Example. Pattern — supply `buffer` and `base`, a multiple of 8 for an 8-pin transfer (§12.0).

```
' Stream bytes to 8 pins
        RDFAST  #0, ##buffer
        DRVL    #7<<6 + base              ' enable those 8 pins (11.0)
        XINIT   ##X_RFBYTE_8P_1DAC8 | X_PINS_ON | (base<<17) | 256, #0
```

Chapter 7: RGB Video Modes

Video earns its own family of modes because pixels are not just bytes. A color pixel must be unpacked into red, green, and blue and pushed out in a form a monitor understands. These RGB modes pull pixel data from a framebuffer and unpack each pixel into full-width red, green and blue on the way to the pins — so your code stores a picture and the streamer turns it into a signal. That unpacking is the streamer's own; the cog's colorspace converter is a separate stage further downstream, and §15.0 covers it. The modes differ mainly in how many bits each pixel uses, trading color depth against memory.

7.1 RGB Format Modes

Mode	Symbol	Hub Read	Format	Bytes/px
%1011_0010	X_RFBYTE_LUMA8	RFBYTE	Luminance 8	1
%1011_0011	X_RFBYTE_RGBI8	RFBYTE	Color 3 + luma 5	1
%1011_0100	X_RFBYTE_RGB8	RFBYTE	RGB 3:3:2	1
%1011_0101	X_RFWORD_RGB16	RFWORD	RGB 5:6:5	2
%1011_0110	X_RFLONG_RGB24	RFLONG	RGB 8:8:8	4

Memory is usually the deciding factor. A framebuffer is `width * height * bytes/px`, and it shares the P2's **512 KB hub RAM** with your code. A full 640×480 frame is **300 KB** at 1 byte/px (fits), **600 KB** at 2 bytes/px, and **1.2 MB** at 4 bytes/px (neither fits). So full-screen video on a 512 KB P2 generally uses a **1-byte format** (RGB8, RGBI8, or LUMA8); RGB16 and RGB24 suit smaller regions, sprites, or boards with external PSRAM.

7.2 Color Format Details

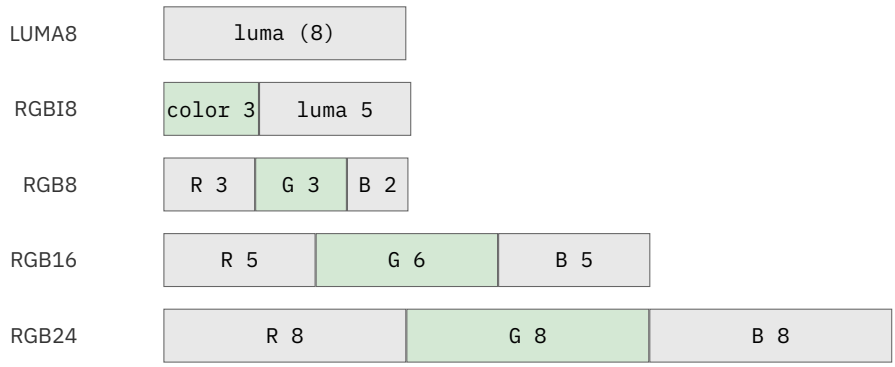


Figure 7.1. Source-format bit packing. LUMA8 and RGBI8 both select one of eight colors and carry a luminance value – LUMA8 takes its color from `S[2:0]`, RGBI8 from the pixel's top three bits. Only RGB8/16/24 carry per-channel values.

LUMA8: 8-bit luminance. The `S[2:0]` field selects one of eight output colors; the pixel byte sets that color's intensity. The color set is fixed:

S[2:0]	Color	S[2:0]	Color
%000	Orange	%100	Red
%001	Blue	%101	Magenta
%010	Green	%110	Yellow
%011	Cyan	%111	White

RGBI8 (color 3 + luminance 5): The top three bits of each pixel, `P[7:5]`, select one of eight output colors — **the same eight LUMA8 offers** — and the bottom five bits, `P[4:0]`, set that color's intensity. This is LUMA8's mechanism with the color select moved out of `S` and into the pixel: that is what buys per-pixel color, and it costs three of the eight luminance bits. There are no separate red, green and blue fields. The five luminance bits are replicated up to fill each driven channel (`P[4,3,2,1,0,4,3,2]`), so a full-scale pixel still reaches full scale.

RGB8 (3:3:2): Three bits red, three bits green, two bits blue. Compact format for 256-color graphics.

RGB16 (5:6:5): Five bits red, six bits green, five bits blue. Standard 65,536-color format.

RGB24 (8:8:8): Eight bits each for R, G, B. True color, one byte wasted per pixel.

7.3 RGB Mode Example

This one routes all four DAC channels (`X_DACS_3_2_1_0`), so the four pins carrying R, G, B and sync must each be configured for DAC output per §11.0 before any of it appears as a voltage. **Pattern** — supply `base` (a multiple of 8, §12.0), `framebuffer` and `cmd`; §15.1 works the same arrangement through as a complete program.

```
' VGA 640×480 RGB16 output (assumes 250 MHz sysclk)
        RDFAST  ##640*480*2/64, ##framebuffer
        SETXFRQ ##$0CCC_CCCC+1                    ' 25 MHz pixel rate

        MOV     cmd, ##X_RFWORD_RGB16 | X_PINS_ON | X_DACS_3_2_1_0
        OR      cmd, ##(base<<17) | 640
        XINIT   cmd, #0  ' XINIT starts from a zeroed phase
```

TIP

RGB16 (`X_RFWORD_RGB16`) costs 2 bytes per pixel, so a full 640×480 frame is 600 KB and does not fit in hub RAM (§7.1). Use it for regions, sprites, or a reduced height; use a 1-byte format for full-screen video on a 512 KB P2.

Chapter 8: WRFAST Input Modes

Here the pipe runs the other way. Instead of driving the pins, these modes *watch* them: on every NCO beat the streamer samples a group of pins and writes the result into hub memory. That turns a cog into a logic analyzer, capturing fast digital activity that software could never sample quickly enough — provided you aim it at pins whose IN carries a logic level, which is what §8.0 is about. The captured data flows out through the write FIFO, which — like its read counterpart — must be primed first.

CAUTION

Run WRFAST before any capture command. It primes that same hub FIFO to *receive* data (streamer → hub); until it does, captured data has no valid destination. This is the exact mirror of RDFAST (Chapter 6) — one FIFO, opposite direction.

8.0 What the Capture Path Actually Reads

Read this before designing a capture. The mechanism is simple and the trap in it is not.

The streamer reads `{INB, INA}` — the cog's input registers — not the pads. For a plain input pin those are the same thing, and everything below is unnecessary. For a pin that is running a smart pin they are not: when a smart pin has a mode-related event it raises its IN signal to tell the cog that data is ready, data can be loaded, or a process has finished. On such a pin, **IN is the smart pin's handshake, not the wire's logic level**, and it is lowered by a `WRPIN`, `WXPIN`, `WYPIN`, `RDPIN` or `AKPIN` on that pin — not by anything the wire does.

CAUTION

Pointing a capture straight at a working SPI, serial or I²C bus records the wrong thing. You get one bit per bus pin that sits low while a transfer is in flight and pulses high at each byte boundary — a transaction-completion trace. It is real data about the smart pin, and it is not the bus.

Nothing errors. The command assembles, the streamer runs, and the hub buffer fills with plausible-looking bytes at exactly the expected rate. The only tell is that the data does not look like the protocol — which, without this rule, reads as a clocking or alignment mistake and costs a long afternoon.

Capturing a pin that is busy being a smart pin

The fix is the mechanism, not a workaround. The `%AAAA` input selector exists so that a pin's logic can take its 'A' input from a neighbour — that is how P2 SPI is wired in the first place, with a data pin taking its clock from the pin next door. Capture uses the same routing in the same direction.

The monitor pin is not a substitute for the bus pin; it is the only pin whose IN can carry the bus pin's level while the bus pin is busy.

1. **Pick a monitor pin within ±3 of the bus pin.** That is the whole reach of the input selector; there is no way to route further.
2. **Configure the monitor with its smart-pin mode off** — `%SSSSS = %00000` — and its A-input selector pointing back at the bus pin. Mode off is the condition under which the resultant 'A' drives IN at all.
3. **Leave the monitor's `DIR` low.** It is an input and must not drive the net it is watching.
4. **`WRFAST` from cog RAM to aim the FIFO at your buffer**, then start a Pins→WRFAST mode with `%w` set and the base pin at the monitor (or at the aligned base of the block containing it).
5. **Read the buffer.** Each captured bit is the bus pin's level as the monitor's IN saw it.

The selector constant names the offset **from the monitor to the bus pin**: a monitor at `bus_pin+1` selects relative −1 (`P_MINUS1_A`), a monitor at `bus_pin-2` selects relative +2 (`P_PLUS2_A`). The set runs `P_MINUS3_A`...`P_PLUS3_A`. `%BBBB` is untouched, so the monitor can still take a second signal on 'B' if something else needs it.

HARDWARE

`DIR` high is the output rule, not the capture rule. §11.0 and §16.1 require `DIRH` because they *drive* pins — that is the streamer's output path. Capture is the input path: the monitor pin's `DIR` stays **low**. Raising it makes your monitor drive the net it is supposed to be watching.

Laying out more than one channel. A four-wire bus needs four monitors, and the streamer captures one contiguous, aligned block. Place the monitors so they form a single aligned block of the width the capture mode needs — four monitors on an address that is a multiple of 4 for a 4-pin mode. Interleaving bus and monitor pins one-for-one wastes half the capture width on pins whose IN carries flags. The alignment is not advisory: §12.0's rule applies here, and an unaligned base neither rounds nor errors.

`%FFF` applies input logic and any global filtering **after** the A/B selectors, and the resultant 'A' is what drives IN. Left at its default `%000` the path is unfiltered; any filter setting inserts flipflops in the sampled path.

TIP

What this guide does not establish. We have no measured ceiling for sustained capture rate, and no figure for whether a filter selection delays one channel enough to skew it against another. Neither is stated here rather than estimated. Displaying a capture — the DEBUG `LOGIC` window, `LONGS_2BIT` unpacking, mask/match triggers — belongs to the *P2 Debug Window Manual*; note only that it offers **no protocol decoders**, so an I²C, SPI, UART or CAN trace is yours to interpret.

8.1 Pin Capture Modes

Mode	Symbol	Pins	DAC Channels	DAC Bits	Hub Write
%1100	X_1P_1DAC1_WFBYTE	1	1	1	WFBYTE
%1101	X_2P_2DAC1_WFBYTE	2	2	1	WFBYTE
%1101	X_2P_1DAC2_WFBYTE	2	1	2	WFBYTE
%1110	X_4P_4DAC1_WFBYTE	4	4	1	WFBYTE
%1110	X_4P_2DAC2_WFBYTE	4	2	2	WFBYTE
%1110	X_4P_1DAC4_WFBYTE	4	1	4	WFBYTE
%1110	X_8P_4DAC2_WFBYTE	8	4	2	WFBYTE
%1110	X_8P_2DAC4_WFBYTE	8	2	4	WFBYTE
%1110	X_8P_1DAC8_WFBYTE	8	1	8	WFBYTE
%1110	X_16P_4DAC4_WFWORD	16	4	4	WFWORD
%1111	X_16P_2DAC8_WFWORD	16	2	8	WFWORD
%1111	X_32P_4DAC8_WFLONG	32	4	8	WFLONG

D[23] = %w: Must be 1 to enable WRFAST writes

HARDWARE

Samples pack before they are written. In the 1-, 2- and 4-pin modes a WFBYTE is done each time **8 bits of pin data accrue** — so a 1-pin mode writes one byte per 8 NCO rollovers, a 2-pin mode one per 4, a 4-pin mode one per 2. Size the buffer from that packed rate, not from the count field, or a 1-pin capture will over-allocate eightfold.

Example. Pattern — supply `capture_buffer` and `base`, a multiple of 8 (§12.0), and size the buffer for 1000 longs.

```
' Capture 32 pins to Hub at 10 MHz
        WRFAST  #0, ##capture_buffer
        SETXFRQ ##$0CCC_CCCC+1

        XINIT   ##X_32P_4DAC8_WFLONG | X_WRITE_ON | (base<<17) | 1000, #0
        WAITXFI
```

Chapter 9: ADC Sampling Modes

ADC modes are the analog cousin of the pin-capture modes in the previous chapter. Instead of recording whether a pin is high or low, they record *how much* — the digitized voltage on an ADC-capable pin. Streaming those readings into memory at a steady rate turns a cog into an oscilloscope or a data logger. Reach for these when you need to capture a waveform, not just a bit.

9.1 ADC Capture Modes

Mode	Symbol	ADCs	Pins	Hub Write
%1111_0010	X_1ADC8_0P_1DAC8_WFBYTE	1	0	WFBYTE
%1111_0011	X_1ADC8_8P_2DAC8_WFWORD	1	8	WFWORD
%1111_0100	X_2ADC8_0P_2DAC8_WFWORD	2	0	WFWORD
%1111_0101	X_2ADC8_16P_4DAC8_WFLONG	2	16	WFLONG
%1111_0110	X_4ADC8_0P_4DAC8_WFLONG	4	0	WFLONG

ADC Pin Requirements: ADC-capable pins must be configured for ADC mode using **WRPIN** before sampling, and the pin must be **enabled** (DIRH) — these modes read a smart pin's ADC result, not a raw bitstream. (The DDS/Goertzel mode of Chapter 10 is the opposite case and takes raw pins; see §17.1.)

These modes take their input from the cog's **four-channel scope**, so the pins are routed with SETSCP rather than named in the streamer command. SETSCP takes the enable in D[6] and a **four-pin block** in D[5:2]. Which of those four channels the command samples then depends on how many it captures: the **1-ADC8** modes take the channel number from S[1:0]; the **2-ADC8** modes take S[1] alone, selecting the upper or the lower pair; and the **4-ADC8** mode captures all four and ignores S.

In the combined modes, pin data occupies the low half of each element and ADC data the high half. X_1ADC8_8P_2DAC8_WFWORD puts the low 8 pins of the %ppp group in the low byte of each word and the one scope channel in the high byte; X_2ADC8_16P_4DAC8_WFLONG puts the low 16 pins in the low half of each long and the two scope channels in the high half. A buffer cannot be decoded without knowing which half is which.

9.2 ADC Configuration Example

Pattern — supply adc_pin, adc_buffer sized for 1024 bytes, and a cmd register.

```
' Route pins 0..3 into the four scope channels and enable the scope
        SETSCP  #%100_0000      ' D[6]=1 enable, D[5:2]=%0000 -> pin base 0

' Configure the ADC pin and ENABLE it
        WRPIN   ##P_ADC_1X | P_ADC_SCOPE, #adc_pin  ' gain + ADC scope mode
        DIRH    #adc_pin

' Capture 1024 ADC samples from scope channel 0
        WRFAST  #0, ##adc_buffer
        MOV     cmd, ##X_1ADC8_0P_1DAC8_WFBYTE | X_WRITE_ON
        ADD     cmd, ##1024             ' count only
        XINIT   cmd, #0                 ' S[1:0] = 0 -> scope channel 0
        WAITXFI
```

HARDWARE

This mode has no pin field — do not add one. `X_1ADC8_0P_1DAC8_WFBYTE` encodes as `%1111_DDDD_W000_0010`, in which `D[22:20]` are **fixed zeros**. Writing a pin number into the command (`adc_pin<<17`, the idiom that is correct in the pin-output modes) lands in `D[19:16]`, and the addition carries into the mode bits — **selecting a different streamer mode entirely**, with no error. The effect is measurable in the transfer size alone: with `adc_pin` = 0 the block above writes 1,024 bytes; with 1 it writes 2,048; with 2 it writes 4,096. It appears to work for `adc_pin` = 0 — and even then it never selected the channel, because the channel comes from `S`.
This is the general rule biting in one place: streamer command fields are positional and mode-specific. §12.0 states it; Appendix A gives the per-mode templates.

HARDWARE

ADC readings are 8-bit values. For higher resolution, use smart pin ADC modes with post-processing.

TIP

Capture-to-spectrum. Streaming ADC samples to hub at up to megasamples per second is the front end of on-chip spectral analysis: capture a block here, then hand it to a CORDIC FFT to turn the samples into a spectrum. The FFT side is worked in the *CORDIC for Real Work* application note (P2AN002); this chapter is how you feed it.

Chapter 10: DDS/Goertzel Mode

This mode does two things at once (Chapter 1 introduced both in plain terms). **DDS** *generates* a signal — it steps through a waveform table to synthesize a precise tone or arbitrary shape. **Goertzel** *measures* one — it reports how much of a single chosen frequency is present in an incoming signal, the trick behind touch-tone decoding and ultrasonic ranging. Uniquely, this mode advances on **every clock cycle**, not just on NCO rollovers, which is what gives it the resolution to do real signal processing.

10.1 Mode Variants

D[31:16]	Symbol	Filter
`%1111_dddd_0ppp_p111`	`X_DDS_GOERTZEL_SINC1`	SINC1
`%1111_dddd_1ppp_p111`	`X_DDS_GOERTZEL_SINC2`	SINC2

That is the whole upper half of the command word, nibble by nibble: mode `%1111`, the DAC routing field `%dddd` (Chapter 11), then D[23] — which alone selects the filter, `0` = SINC1 and `1` = SINC2 — and then the four-pin input block.

The block selector straddles a nibble boundary, and that is worth seeing. It is `%pppp` in D[22:19]: three of its bits sit beside D[23], and its low bit lands in D[19], immediately above the fixed `%111`. Appendix A prints only D[19:16] and so shows this row as `%p111` — the same field, seen through a narrower window. §13.4 works through why a selector spanning D[22:19] is what makes `base<<17` require a multiple of four.

10.2 Operation

On each system clock:

1. NCO phase selects a LUT entry — `LUT[NCO[30:22]]` at the default window size of 512. The size is selectable, and the index bits move with it (§10.3).
2. NCO phase advances: `NCO += frequency`
3. LUT bytes output to DACs (XOR `$80` for unsigned)
4. ADC input multiplied by sine/cosine from LUT
5. Products accumulated in sine/cosine registers

```
DAC3 := LUT.BYTE[3] ^ $80
DAC2 := LUT.BYTE[2] ^ $80
DAC1 := LUT.BYTE[1] ^ $80
DAC0 := LUT.BYTE[0] ^ $80

sin := LUT.BYTE[3] (sign-extended)
cos := LUT.BYTE[2] (sign-extended)
m := bitstream sum, -3 to +3 (±1 per selected ADC pin)

sin_acc += sin × m
cos_acc += cos × m
```

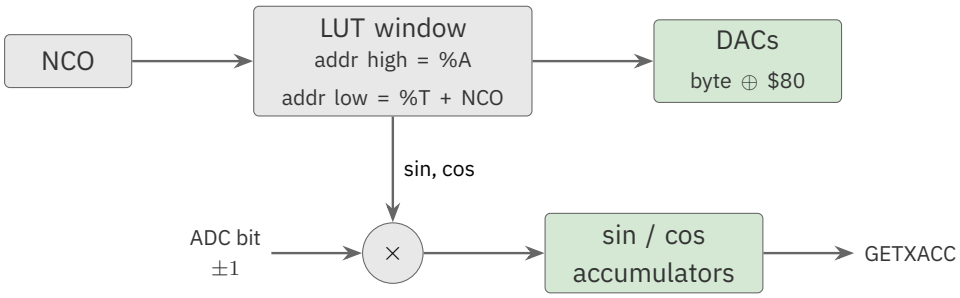


Figure 10.1. DDS output with Goertzel accumulation (every clock). The window is 4 to 512 entries, selected by S[11:9]; %A bounds it and %T offsets playback within it.

10.3 The LUT Window

§10.2 gave the index as `LUT[NCO[30:22]]`. That is one case of eight. `S` carries three things: `S[19:16]` selects which block pins are **inverted**, `S[15:12]` selects which are **summed** — leave it zero and the analyzer accumulates zero, which §17.1 works through — and `S[11:0]`, the subject of this section, selects **how much** of the lookup RAM the NCO walks, **which part** of it, and **where in that part playback starts** — and the last of those is the field that performs the modulation this chapter's applications advertise.

The top three bits pick the loop size. The nine bits below them split into region bits `%A` and offset bits `%T`, and the split moves as the loop size changes:

S[11:0]	Loop size	NCO bits	LUT range
%000_TTTTTTTTT	512	30..22	%000000000..%111111111
%001_ATTTTTTTT	256	30..23	%A00000000..%A11111111
%010_AATTTTTTT	128	30..24	%AA0000000..%AA1111111
%011_AAATTTTTT	64	30..25	%AAA000000..%AAA111111
%100_AAAATTTTT	32	30..26	%AAAA00000..%AAAA11111
%101_AAAAATTTT	16	30..27	%AAAAA0000..%AAAAA1111
%110_AAAAAATTT	8	30..28	%AAAAAA000..%AAAAAA111
%111_AAAAAAATT	4	30..29	%AAAAAAA00..%AAAAAAA11

Read a row downward and one pattern runs through all eight: each step halves the loop, drops one NCO bit out of the index, and hands the bit it freed to %A. The bits are conserved — the LUT address is always nine bits wide, because the lookup RAM is always 512 longs. What changes is how many of those nine the NCO supplies and how many you supply.

%A — which region. The %A bits are the high bits of the address, and they hold still. A loop size of 64 walks 64 consecutive longs, and %AAA says which of the eight 64-long regions. So the LUT can hold eight different 64-entry waveforms at once and a command selects one, with no reloading.

%T — where playback starts, and how it moves. On each clock the lookup RAM is read at the nine-bit location bound by the %A bits, with the lower bits being the sum of the %T bits and the topmost NCO bits. %T is therefore an offset added to the NCO's position within the window: one step of %T shifts playback by one entry, and the %A bits bound the result, so the offset moves within the window rather than out of it.

Because S is a command operand, each streamer command carries its own %T. Two commands over the same waveform with different %T values play it from different phases — which is what "shift or modulate the phase of playback" means in practice, and why a chapter whose applications include RF modulation needs this field and not just the default.

CAUTION

512 is the default, not a requirement. A loop size of 512 (S[11:9] = %000) leaves all nine index bits to the NCO and no %A bits at all, so the window is the whole LUT and there is nothing to position. Every smaller size trades NCO index bits for placement. Code written as though 512 were mandatory still works — it is simply using one of the eight settings.

10.4 LUT Setup

A full-window table holds 512 entries of signed sine/cosine values. A smaller loop size needs only as many entries as its window (§10.3), placed in the region its %A bits select:

Pattern — supply the `sine_table` array of 512 longs and the `cos`, `sin`, `t` and `i` locals.

```
' Build the sine/cosine table in a hub array, then bulk-load it to LUT
repeat i from 0 to 511
  cos, sin := polxy(127, i << 23)
  t.BYTE[3] := sin          ' Sine for Goertzel
  t.BYTE[2] := cos          ' Cosine for Goertzel
  t.BYTE[1] := 0            ' Unused
  t.BYTE[0] := sin          ' Optional DAC output
  sine_table[i] := t        ' loaded to LUT in §17.1 via SETQ2+RDLONG
```

10.5 SINC1 vs SINC2

Characteristic	SINC1	SINC2
Accumulation	Direct	Double integration
Q factor	Lower	Higher
Selectivity	Broader	Sharper
Max amplitude	±127 (full signed byte)	±10 (prevents overflow)

CAUTION

SINC2 double-integrates, so its accumulators grow far faster than SINC1's. Scale the LUT waveform amplitude to about ±10 (the value the Goertzel example in the *Parallax Propeller 2 Documentation v35 - Rev B/C* uses for SINC2) to prevent accumulator overflow.

CAUTION

SINC2 requires a *constant* iteration count per Goertzel cycle — a silicon limitation reported by the P2's designer. SINC2's double integration is only correct when every Goertzel cycle integrates the same number of streamer iterations. If the NCO frequency word (`SETXFRQ`'s D) makes one NCO cycle span a non-power-of-two number of system clocks, the iteration count varies by ±1 clock from cycle to cycle; GETXACC then captures an accumulator that is off by one integration, corrupting the current sample **and the following one** before it self-corrects. The symptom is periodic noise in the output. (Chip Gracey, the P2's designer, reported this constraint on 2024-12-16; it is stated in the *Parallax Propeller 2 Documentation*, in its note of that date on Goertzel SINC2 mode.)
Three ways to avoid it, most robust first:

1. **Run at a system clock that makes the iteration count a power of two.** To listen at 1 MHz, run the sysclock at 256 MHz rather than 250 MHz, so every Goertzel cy-

continues on next page →

↪ continued from previous page

cle is exactly 256 clocks — constant by construction. (A *constant* count is the true requirement; a power of two is simply the practical way to guarantee it.)

2. **Use SINC1 instead.** Single integration is not sensitive to a varying iteration count.
3. **If you must use SINC2 with a non-power-of-two rate, start each measurement with XZERO (not XCONT).** Measured on P2 silicon, a corrupted pair occurs at every change of window length and never otherwise; started with XZERO, each measurement held one window length and gave clean samples at 10.24 µs, 100 µs and 25 ms windows.

10.6 Reading Results

`GETXACC` returns the cosine accumulator in `D`, and the sine half arrives in the **next instruction's S operand** — which is what the `0-0` placeholder below receives. Rev C The read clears the accumulators only while a Goertzel command is running; with the streamer idle it returns the running total and clears nothing, so a second idle read returns the same numbers. §17.1 works the consequence through.

Pattern — supply the `cos_result`, `sin_result`, `magnitude` and `phase` registers.

```
        GETXACC cos_result          ' Cosine accumulator → D
        MOV     sin_result, 0-0     ' Sine accumulator → next S

        QVECTOR cos_result, sin_result
        GETQX   magnitude
        GETQY   phase
```

10.7 Frequency Calculation

To detect frequency F at clock rate CLK:

```
frequency = $8000_0000 × F / CLK
```

HARDWARE

The Goertzel NCO uses the same SETXFRQ scaling as every streamer mode — the multiplier is `$8000_0000` (2^{31}), because the NCO masks its MSB each clock. Resolution is `clock_frequency / 2^31`, about 0.12 Hz at 250 MHz.

Part III: Configuration Reference

These chapters cover the choices that apply across modes — where data goes among the DAC channels, which pins are driven, how commands are named, and how your code stays in step with the streamer.

Chapter 11: DAC Channel Configuration

Many modes send data to the DAC channels, but none of them say *which* channels, or *how*. That is this chapter's job. The %dddd routing field is the knob from Chapter 1's stereo example: it decides how the streamer's data spreads across the four 8-bit DAC channels — one channel, a stereo pair, a differential pair, or all four independently. The same data becomes mono, stereo, or four-channel purely by changing this field.

11.0 Getting a DAC Channel Onto a Pin

The routing field decides which of the four cog DAC channels the streamer's data lands on. It does not put any of them on a pin. That is a separate step, taken on the pin rather than in the command, and every DAC example in this book assumes it has been done.

The requirement is stated plainly by the hardware documentation: to bring the data out as a voltage on a pin, that pin must be set to DAC mode with the COGID embedded, via `WRPIN`, and `DIR` must be set high. Three things, and all three are needed:

- **DAC mode** — `WRPIN` with a DAC pin-mode constant, which also fixes the drive impedance and full-scale voltage. Four are available: `P_DAC_990R_3V` (990 Ω, 3.3 V peak), `P_DAC_600R_2V` (600 Ω, 2.0 V), `P_DAC_124R_3V` (123.75 Ω, 3.3 V), and `P_DAC_75R_2V` (75 Ω, 2.0 V). Pick for the load being driven.
- **The COGID**, in `M[3:0]` — which cog's DAC channels this pin listens to. Cogs each have their own set of four.
- **`DIRH`** on the pin. Until DIR is high, the pin does not drive.

Which of the four channels a pin takes is decided by the pin, not by the command. A pin's low two bits select its channel — pin %xxxx00 takes DAC0, %xxxx01 takes DAC1, and so on. §11.2 gives the mapping. This is why `M[3:0]` has room for a cog number: the channel is already chosen by the time the mode word is read.

Two arrangements, and the same constant inverts between them. The %TT field in the pin's mode word decides where the DAC's value comes from, and DAC mode gives it two meanings that are opposites:

%TT	Source of the DAC value	Constant
%00	the pin's **own level field**, M[7:0], written by WRPIN	*(none — this is the default)*
%01	a **cog DAC channel**, which is what SETDACS and the streamer write	P_CHANNEL

A streamer-driven DAC is always the second case: the streamer writes cog DAC channels, so the pin must be listening to one. A level-driven DAC is the first case, and adding P_CHANNEL to it **kills its output** — it points the pin at a channel instead of at the level you just wrote. Same constant, opposite effect, decided by who supplies the value. (P_CHANNEL, P_OE and P_TT_01 are one bit-field value under three names; §13.4 explains why they must be combined with | and never +.)

```
CON   DAC_PIN = 8            ' low bits %00, so this pin takes DAC0

DAT             ORG
                COGID   cogn                    ' this cog, 0..7
                SETNIB  dacmode, cogn, #2       ' COGID -> M[3:0]
                WRPIN   dacmode, #DAC_PIN       ' DAC mode + cog source
                DIRH    #DAC_PIN                ' the pin now drives

dacmode         LONG    P_DAC_124R_3V | P_CHANNEL
cogn            RES     1
```

SETNIB writes nibble 2 of the mode word, which is M[3:0] — the WRPIN operand is laid out as %AAAA_BBBB_FFF_MMMMMMMMMMMM_TT_SSSSS_0, so the M field's low nibble sits at bits 11..8.

What the channel holds when the streamer is not driving it. SETDACS sets the background value of all four channels at once — it writes bytes 3, 2, 1 and 0 of its operand to DAC3, DAC2, DAC1 and DAC0. Those values are output continuously, *except* while the streamer or the colorspace converter overrides them. That is exactly what the -- entries in §11.1's routing table mean: a channel the streamer does not override keeps emitting its SETDACS value. A routing choice such as X_DACS_X_X_1_0, which drives only DAC1 and DAC0, leaves DAC3 and DAC2 sitting at whatever SETDACS last put there — silence if you set it, and whatever was left over if you did not.

```
                SETDACS ##$80_80_80_80          ' all four channels to
                                                ' mid-scale: the rest
                                                ' level for a signed
                                                ' waveform (see 10.2)
```

HARDWARE

Digital pin output needs less of this — not none of it. Ordinary pin output through X_PINS_ON requires no WRPIN: no DAC mode, no COGID, no channel. The mode configuration above is for **DAC** output only, and adding it to a digital-output example is a different mistake, not a safer one.

What digital output still needs is DIRH. X_PINS_ON enables the streamer's contribution to the pin's output *state* — the same signal OUT supplies — and DIR remains what enables the pin to drive at all. Until DIR is high, the pin does not drive, exactly as in the list above. A streamer command aimed at a pin whose DIR is low runs to completion and changes nothing: the count decrements, WAITXFI returns, and the pin never moves. On the bench that reads as a pin holding whatever charge it had, which looks like noise rather than like a missing step.

11.1 DAC Routing Table

%dddd	DAC3	DAC2	DAC1	DAC0	Symbol
%0000	–	–	–	–	X_DACS_OFF
%0001	X0	X0	X0	X0	X_DACS_0_0_0_0
%0010	–	–	X0	X0	X_DACS_X_X_0_0
%0011	X0	X0	–	–	X_DACS_0_0_X_X
%0100	–	–	–	X0	X_DACS_X_X_X_0
%0101	–	–	X0	–	X_DACS_X_X_0_X
%0110	–	X0	–	–	X_DACS_X_0_X_X
%0111	X0	–	–	–	X_DACS_0_X_X_X
%1000	!X0	X0	!X0	X0	X_DACS_0N0_0N0
%1001	–	–	!X0	X0	X_DACS_X_X_0N0
%1010	!X0	X0	–	–	X_DACS_0N0_X_X
%1011	X1	X0	X1	X0	X_DACS_1_0_1_0
%1100	–	–	X1	X0	X_DACS_X_X_1_0
%1101	X1	X0	–	–	X_DACS_1_0_X_X
%1110	!X1	X1	!X0	X0	X_DACS_1N1_0N0
%1111	X3	X2	X1	X0	X_DACS_3_2_1_0

Legend:

- `--` = No override — the channel keeps emitting its `SETDACS` background value (§11.0)
- `!` = One's complement (inverted)
- `X0`-`X3` = streamer data channels

Every routing choice here still needs the pin-side setup of §11.0 before any of it reaches a voltage.

11.2 DAC Pin Mapping

DAC channels drive pins based on the pin's two LSBs:

DAC Channel	Pin Pattern	Example Pins
DAC0	`%xxxx00`	0, 4, 8, 12, 16...
DAC1	`%xxxx01`	1, 5, 9, 13, 17...
DAC2	`%xxxx10`	2, 6, 10, 14, 18...
DAC3	`%xxxx11`	3, 7, 11, 15, 19...

HARDWARE

Each DAC channel can only drive pins matching its channel number in the two LSBs. This is a silicon constraint, not a configuration option.

11.3 Common DAC Configurations

These are mode words, not programs: each shows the routing choice for one arrangement. The pins they drive must be configured per §11.0 first, and `SETDACS` decides what any channel the routing leaves at `--` emits.

Mono Audio (single channel). Pattern — supply `pin` and `count`.

```
mode := X_RFBYTE_1P_1DAC1 | X_DACS_0_0_0_0 | X_PINS_ON + pin<<17 + count
```

Stereo Audio (two channels). Pattern — supply `pin` and `count`. DAC3 and DAC2 are left at their `SETDACS` values.

```
mode := X_RFWORD_16P_2DAC8 | X_DACS_X_X_1_0 | X_PINS_ON | (pin<<17) | count
```

Differential Output (noise rejection). Pattern — supply `pin` and `count`. Both pins of the pair need §11.0 setup, and their low two bits must select DAC1 and DAC0.

```
    mode := X_RFBYTE_1P_1DAC1 | X_DACS_X_X_0N0 | X_PINS_ON + pin<<17 + count
```

Four-Channel Video (RGB + sync). Pattern — supply `pin` and `count`. All four pins need §11.0 setup; §15.1 works this arrangement through completely.

```
    mode := X_RFLONG_32P_4DAC8 | X_DACS_3_2_1_0 | X_PINS_ON | (pin<<17) | count
```

Chapter 12: Pin Selection and Control

A streamer command also has to say *which* pins it drives or samples, and that is less obvious than it sounds: the P2 has 64 pins, but a command addresses them 32 at a time, through a window you choose. This chapter covers how to aim the streamer at the right pins, how to enable output, and a few smaller controls such as bit ordering.

12.0 Reading a Pin Field

Two rules govern every pin field in this chapter. Both are easier to state here, once, than to rediscover per mode.

Streamer command fields are positional and mode-specific. A field that exists in one mode is fixed or absent in another, so an idiom carried across modes is not portable — it does not fail loudly, it selects something else. Chapter 4 gives the layout; Appendix A gives the per-mode templates. §9.2 works through a case where writing a pin number into a mode that has no pin field silently changed the transfer size.

The `pin<<17` idiom, and why it works. The pin-group field sits at `D[22:20]` and selects the window in 8-pin increments; the bits below it, `D[19:17]`, resolve the pin within that window for transfers of fewer than eight pins. A single shift therefore splits a pin number across both fields: `pin<<17` puts `pin>>3` into the group field and `pin&7` into the sub-pin field, which is exactly the decomposition the two fields expect. That is why the idiom appears throughout this book with a plain pin number.

CAUTION

The shift is arithmetic, not a pin-field operator. `pin<<17` is correct only when the low bits it lands in are pin bits *for that mode*. In the fewer-than-8-pin modes some of `D[19:17]` are DAC-configuration bits rather than pin bits (§12.2 gives the split per pin count), and in DDS/Goertzel the field is `D[22:19]` holding a four-pin block number — where `base<<17` sets that field correctly only when `base` is a multiple of four (§13.4). Check the field before reusing the shift.

At eight pins and wider, `D[19:17]` holds no pin bits at all, so the operand must be a **multiple of 8** — a window base, not an arbitrary pin. An unaligned value there does not merely land on the wrong pins. Its low three bits fall into the mode's own `D[19:16]` template, and because the idiom composes with `+`, they *carry*: a 640-pixel `X_IMM_4X8_1DAC8` written with `pin = 20` assembles to a **different mode** (`X_IMM_4X8_4DAC2`) driving a **different window** (pins 31..24, not 23..16). Nothing warns you — both forms are legal arithmetic and both compile.

The alignment rule, in full. The base must be a multiple of the mode's pin width: **any pin** for the 1-pin modes, an **even** pin for 2-pin, a multiple of **4** for 4-pin, and a multiple of **8**

continues on next page →

↪ *continued from previous page*

for 8-pin and wider. DDS/Goertzel is the separate case above — a four-pin block number in `D[22:19]`, so a multiple of four.

Compose with `|`, and only after checking the alignment. The two operators fail differently, and neither one rescues a base that is misaligned:

	What an unaligned base does
`+`	**carries** into `D[19:16]` — a different mode, at a different pin group
`|`	cannot carry, so the stray bits set bits the template already sets and the base **silently vanishes** — the transfer runs at the aligned address instead

So `|` is the *safer* operator and not a safe one: with `+` you get the wrong thing, with `|` you get the right thing for the wrong reason and learn nothing. Check the alignment first; then compose with `|` so a mistake cannot change the mode.

12.1 Pin Group Selection

The %ppp field in D[22:20] selects the 32-pin window the streamer drives or samples. The window's **base pin is `%ppp * 8`**, and it always spans **32 consecutive pins** from there — wrapping past pin 63 back to pin 0 once the base climbs high enough.

%ppp	Base	Pin Range (always 32 pins)	Window
%000	0	31..0	low pins
%001	8	39..8	shifted up 8
%010	16	47..16	middle
%011	24	55..24	shifted up 24
%100	32	63..32	high pins
%101	40	63..40, 7..0	wrap (24 + 8)
%110	48	63..48, 15..0	wrap (16 + 16)
%111	56	63..56, 23..0	wrap (8 + 24)

The **wrap-around groups (%101–%111)** place a 32-pin window that straddles the top and bottom of the pin field — useful when the pins for one function sit at both ends of the chip (for example a peripheral wired across 63..40 and 7..0).

CAUTION

A wrap-around range is still 32 pins, not fewer. "`63..40, 7..0`" reads as two fragments but means pins 63 down to 40 (24 pins) *plus* 7 down to 0 (8 pins) = **32 total**. Don't mistake the split notation for a smaller window.

12.2 Sub-Pin Selection

Within the 32-pin window chosen by the group field (§12.1), the D[19:17] region refines *which* pins a fewer-than-8-pin transfer uses. It is **not** a uniform 3-bit selector across all pin counts: as the pin count rises, fewer of these bits are pin-select bits and the freed low bits become **DAC-configuration** bits (which DACs the block feeds). The pin offset is relative to the window base.

1-Pin modes — all three bits (D[19:17]) select the pin offset (0–7):

D[19:17]	Pin offset
%000	Pin 0
%001	Pin 1
%010	Pin 2
%011	Pin 3
%100	Pin 4
%101	Pin 5
%110	Pin 6
%111	Pin 7

2-Pin modes — only D[19:18] select the pin pair; **D[17] is part of the mode template**, not a pin bit — it is what distinguishes `X_RFBYTE_2P_2DAC1` from `X_RFBYTE_2P_1DAC2`, so writing into it selects a different mode:

D[19:18]	Pin pair
%00	Pins 1..0
%01	Pins 3..2
%10	Pins 5..4
%11	Pins 7..6

4-Pin modes — only D[19] selects the pin group; **D[18:17] are mode-template bits** (4DAC1 / 2DAC2 / 1DAC4), not pin bits — writing into them selects a different mode:

D[19]	Pin group
%0	Pins 3..0
%1	Pins 7..4

HARDWARE

Reach higher pins with the group field, not the sub-pin field. Sub-pin selection only refines within the low pins of the window — it does not scan across all 32. To place a transfer on higher pins, move the 32-pin window with the group field `%ppp` in D[22:20] (§12.1).

12.3 Enable Control

Output Modes. D[23] must be 1 to drive pins. **Pattern** — supply `pin` and `count`.

```
' Pin output enabled
mode := X_RFBYTE_8P_1DAC8 | X_PINS_ON | (pin<<17) | count

' Pin output disabled (DACs only)
mode := X_RFBYTE_8P_1DAC8 | X_PINS_OFF | (pin<<17) | count
```

Input Modes. D[23] must be 1 to write to hub. **Pattern** — supply `pin` and `count`.

```
' WRFAST enabled
mode := X_32P_4DAC8_WFLONG | X_WRITE_ON | (pin<<17) | count

' WRFAST disabled (DACs only)
mode := X_32P_4DAC8_WFLONG | X_WRITE_OFF | (pin<<17) | count
```

12.4 Alternate Bit Order

The %a bit in D[16] controls bit ordering for 1/2/4-bit modes:

D[16]	Order	Symbol
0	LSB first (default)	X_ALT_OFF
1	MSB first	X_ALT_ON

TIP

Use MSB-first (`X_ALT_ON`) for SPI protocols that transmit MSB first — **but only in the 1-, 2- and 4-bit-per-sample modes**. The `%a` bit reorders bits *within* a sub-byte group, and an 8-bit-per-transfer capture has no such grouping, so `X_ALT_ON` does nothing to it. In an 8-bit mode, reorder after the fact (`REV` or `MOVBYTS`) or pick a mode whose grouping `%a` reaches.

Chapter 13: Programming Constants

You rarely build a command word bit by bit. Instead you OR together named constants, such as `X_RFWORD_RGB16`, `X_PINS_ON` and `X_DACS_3_2_1_0`, and the compiler assembles the value for you. This chapter covers the symbols this book uses and shows how they compose; Appendix A and Appendix B carry the complete set. Skim it once to learn the naming pattern; after that the names read almost like sentences.

13.1 Mode Symbols

Immediate → LUT → Pins/DACs:

Symbol	Value	Description
X_IMM_32X1_LUT	%0000 << 28	32×1-bit → LUT
X_IMM_16X2_LUT	%0001 << 28	16×2-bit → LUT
X_IMM_8X4_LUT	%0010 << 28	8×4-bit → LUT
X_IMM_4X8_LUT	%0011 << 28	4×8-bit → LUT

Immediate → Pins/DACs:

Symbol	Value	Description
X_IMM_32X1_1DAC1	%0100 << 28	32×1-bit → 1 pin, 1 DAC ch × 1 bit
X_IMM_16X2_2DAC1	%0101 << 28	16×2-bit → 2 pins, 2 DAC ch × 1 bit
X_IMM_16X2_1DAC2	%0101 << 28 + 2<<16	16×2-bit → 2 pins, 1 DAC ch × 2 bits
X_IMM_8X4_4DAC1	%0110 << 28	8×4-bit → 4 pins, 4 DAC ch × 1 bit
X_IMM_8X4_2DAC2	%0110 << 28 + 2<<16	8×4-bit → 4 pins, 2 DAC ch × 2 bits
X_IMM_8X4_1DAC4	%0110 << 28 + 4<<16	8×4-bit → 4 pins, 1 DAC ch × 4 bits

RDFAST → Pins/DACs:

Symbol	Value	Description
X_RFBYTE_1P_1DAC1	%1000 << 28	RFBYTE, 1-pin
X_RFBYTE_2P_2DAC1	%1001 << 28	RFBYTE, 2-pin
X_RFBYTE_4P_4DAC1	%1010 << 28	RFBYTE, 4-pin
X_RFBYTE_8P_1DAC8	%1010 << 28 + $E<<16	RFBYTE, 8-pin
X_RFWORD_16P_4DAC4	%1010 << 28 + $F<<16	RFWORD, 16-pin
X_RFWORD_16P_2DAC8	%1011 << 28	RFWORD, 16-pin
X_RFLONG_32P_4DAC8	%1011 << 28 + 1<<16	RFLONG, 32-pin

RDFAST → RGB:

Symbol	Value	Description
X_RFBYTE_LUMA8	%1011 << 28 + 2<<16	Color from S, luma 8
X_RFBYTE_RGBI8	%1011 << 28 + 3<<16	Color 3 + luma 5
X_RFBYTE_RGB8	%1011 << 28 + 4<<16	RGB 3:3:2
X_RFWORD_RGB16	%1011 << 28 + 5<<16	RGB 5:6:5
X_RFLONG_RGB24	%1011 << 28 + 6<<16	RGB 8:8:8

DDS/Goertzel:

Symbol	Value	Description
X_DDS_GOERTZEL_SINC1	%1111 << 28 + 7<<16	SINC1 filter
X_DDS_GOERTZEL_SINC2	%1111 << 28 + 7<<16 + 1<<23	SINC2 (D[23]=1)

13.2 Control Symbols

Symbol	Value	Effect
X_PINS_OFF	%0 << 23	Disable pin output
X_PINS_ON	%1 << 23	Enable pin output
X_WRITE_OFF	%0 << 23	Disable WRFAST
X_WRITE_ON	%1 << 23	Enable WRFAST
X_ALT_OFF	%0 << 16	LSB first
X_ALT_ON	%1 << 16	MSB first

X_PINS_ON and X_WRITE_ON are the **same bit**. D[23] is one enable whose meaning follows the mode — pin output in the output modes, WRFAST in the capture modes — so they are not two independent controls, and setting “both” sets one bit once.

13.3 DAC Symbols

Symbol	Value	Configuration
X_DACS_OFF	%0000 << 24	No DAC output
X_DACS_0_0_0_0	%0001 << 24	X0 on all channels
X_DACS_X_X_0_0	%0010 << 24	X0 on channels 0,1
X_DACS_0_0_X_X	%0011 << 24	X0 on channels 2,3
X_DACS_X_X_X_0	%0100 << 24	X0 on channel 0
X_DACS_X_X_0_X	%0101 << 24	X0 on channel 1
X_DACS_X_0_X_X	%0110 << 24	X0 on channel 2
X_DACS_0_X_X_X	%0111 << 24	X0 on channel 3
X_DACS_0N0_0N0	%1000 << 24	Differential pairs
X_DACS_X_X_0N0	%1001 << 24	Diff on 0,1
X_DACS_0N0_X_X	%1010 << 24	Diff on 2,3
X_DACS_1_0_1_0	%1011 << 24	Stereo pairs
X_DACS_X_X_1_0	%1100 << 24	Stereo on 0,1
X_DACS_1_0_X_X	%1101 << 24	Stereo on 2,3
X_DACS_1N1_0N0	%1110 << 24	Differential stereo
X_DACS_3_2_1_0	%1111 << 24	All 4 independent

13.4 Symbol Composition

Build complete commands by combining symbols:

Pattern — supply `vga_base`, `spi_pin`, `adc_base` and `cycles`.

```
' VGA 640-pixel visible line
mode := X_RFWORD_RGB16 | X_PINS_ON | X_DACS_3_2_1_0 | (vga_base<<17) | 640

' SPI byte output (MSB first)
mode := X_IMM_32X1_1DAC1 | X_PINS_ON | X_ALT_ON + spi_pin<<17 + 8

' Goertzel — adc_base is a FOUR-PIN BLOCK, must be a multiple of 4
mode := X_DDS_GOERTZEL_SINC1 | X_DACS_0N0_0N0 + adc_base<<17 + cycles
```

In that last line `adc_base<<17` is shorthand for the block field `D[22:19]`, and it is only correct because `(adc_base>>2)<<19` equals `adc_base<<17` **exactly when `adc_base` is a multiple of four**. The field names a block of four pins, not a pin — §17.1 works through what that means for the input. This is the DDS/Goertzel case of the shift rule in §12.0: the same `<<17` reaches a different field here, so the alignment it needs is different too.

Combine pin-mode constants with |, never +

The `P_*` constants that configure a pin through `WRPIN` are **bit fields positioned inside the mode word**, not additive flags. Constants drawn from the same "pick one" group occupy the **same bits**, so `+` carries out of the field and lands in a neighbouring mode, while `|` sets the field and is idempotent. There is no assembler error and no warning — the pin simply does something else.

The clearest case is the `%TT` field at bits 7:6, because three of its names look like three separate features:

Fragment — the composition alone; supply the `mode` variable that receives the word.

```
' Correct — the field is set to %01 and stays there
mode := P_CHANNEL | P_OE          ' %01 = P_TT_01
```

```
' Wrong — the carry lands in the next mode up
mode := P_CHANNEL + P_OE          ' %10 = P_BITDAC
```

`P_TT_01`, `P_OE` and `P_CHANNEL` are **one bit-field value** (`$40`, `%01`) under three context names — not three features, and not additive. Each name reads correctly in its own context: `P_OE` where a smart pin's output is being enabled, `P_CHANNEL` where a non-smart-pin DAC's source is being selected, `P_TT_01` for the raw field value. A reader who meets all three in the symbol list will assume three independent capabilities; there is one bit.

The failure is silent and total. Measured on P2 silicon, the `|` form drove a cog DAC at 6,737 ADC counts while the `+` form read 1,407 — indistinguishable from no drive at all.

Numeric fields are a separate matter. `adc_pin<<17` and `cycles` above are values shifted into their own positions in the streamer command word, so `+` and `|` agree — provided the field exists in the mode being built and the value fits it. Chapter 4 gives the command word's field layout, which varies by mode.

Chapter 14: Events and Synchronization

Because the streamer runs on its own, your code needs a way to ask *where it is up to* — is it ready for another command, has it finished, did the NCO just roll over? The streamer raises events for exactly these moments, and this chapter shows how to poll them, wait on them, or branch on them. Getting this right is how you chain commands seamlessly and keep video and audio free of glitches.

14.1 Streamer Events

Event #	Symbol	Trigger Condition
10	EVENT_XMT	Streamer ready for new command
11	EVENT_XFI	Streamer finished (no pending command)
12	EVENT_XRO	NCO rollover occurred
13	EVENT_XRL	LUT address $1FF read

14.2 Event Instructions

Polling (non-blocking):

Instruction	Effect
POLLXMT WC	C = 1 if ready for command
POLLXFI WC	C = 1 if finished
POLLXRO WC	C = 1 if NCO rolled over
POLLXRL WC	C = 1 if LUT $1FF read

Waiting (blocking):

Instruction	Effect
WAITXMT	Wait until ready for command
WAITXFI	Wait until finished
WAITXRO	Wait until NCO rollover
WAITXRL	Wait until LUT $1FF read

Conditional Jumps:

Instruction	Condition
`JXMT label`	Jump if ready
`JNXMT label`	Jump if not ready
`JXFI label`	Jump if finished
`JNXFI label`	Jump if not finished
`JXRO label`	Jump if rollover
`JNXRO label`	Jump if no rollover
`JXRL label`	Jump if LUT $1FF
`JNXRL label`	Jump if not LUT $1FF

14.3 Event Clearing

The three streamer-command events — **EVENT_XMT** (10), **EVENT_XFI** (11), and **EVENT_XRO** (12) — clear automatically on:

- **XINIT**, **XCONT**, **XZERO** execution (these instructions re-arm the events)
- **POLL**, **WAIT**, or **J** instruction execution for that event

EVENT_XRL (13, LUT address $1FF read) is the exception: it is **not** re-armed by **XINIT**, **XCONT**, or **XZERO**. It clears only on cog start or on its own poll/wait/jump (POLLXRL/WAITXRL/JXRL/JNXRL).

14.4 Synchronization Patterns

Wait for completion. Pattern — supply `mode` and `data`.

```
        XINIT   mode, data
        WAITXFI                 ' Block until done
```

Chain commands without gaps. Pattern — supply the three `mode` and `data` pairs.

```
        XINIT   mode1, data1    ' Start first command
        XCONT   mode2, data2    ' Queue second command
        XCONT   mode3, data3    ' Queue third command
        WAITXFI                 ' Wait for all to finish
```

Video line timing. Pattern — supply the four interval mode words and `sync_data`; §15.1 works this loop through completely.

```
line    XZERO   m_sync, sync_data   ' Sync pulse (phase zeroed)
        XCONT   m_back, #0          ' Back porch
        XCONT   m_visible, #0       ' Visible pixels
        XCONT   m_front, #0         ' Front porch
        JMP     #line
```

TIP

Use **XZERO** at line start to prevent phase accumulation errors over many lines.

14.5 Debugging Streamer Code

Compiling with `-d` puts the P2's **highest-priority interrupt** inside your streaming cog, and by default it does so in *every* cog: `DEBUG_COGS` defaults to `%1111_1111`. The debug interrupt is not aware of the streamer, so it can preempt a cog mid-transfer — and a streamer transfer that is interrupted does not resume where a reader would expect it to.

The symptom is corrupted data, not a stopped program. In our own Goertzel measurements the streaming cog crashed into the single-step debugger's memory dump and the accumulators read between 1,000,000 and 7,000,000, where the true values were in the hundreds — numbers large enough, stable enough, and plausible enough to be believed. Every measurement taken before the mask was narrowed was confounded.

The fix is one `CON` line: report from the cog you are actually watching, and leave the streaming cog out of the mask.

```
CON
  DEBUG_COGS = %0000_0001    ' debug cog 0 only; streaming cog undisturbed
```

The general rule reaches past the streamer: **any hardware sequencer you are measuring should be running in a cog the debugger is not interrupting.** Smart pins, the CORDIC pipeline and the FIFO are all timing-sensitive in the same way. If adding DEBUG changes your numbers, the DEBUG is part of the measurement.

Part IV: Applications

Here the modes come together into the things people actually build: video, high-speed serial, signal processing, and the patterns that combine them.

Chapter 15: Video Output

Part IV puts the pieces together into real applications, beginning with the streamer's signature use: video. This chapter walks through generating VGA, HDMI, and composite signals — combining the RGB modes of Chapter 7, the NCO timing of Chapter 3, and the sync discipline that keeps a picture stable. The encoding differs by standard, but the shape is always the same: stream a framebuffer, on the beat, line after line. Composite video also depends on a piece of hardware no earlier chapter has needed — the cog's colorspace converter — and HDMI is switched on through the same mode register, so §15.0 covers both first.

15.0 The Colorspace Converter

Composite video is produced by hardware the earlier chapters have not described. Each cog has a **colorspace converter**: a pipeline between that cog's four DAC channels and its pins, performing a matrix transformation and a modulation on every clock. Composite uses both. VGA as §15.1 streams it uses neither — the RGB modes have already put red, green and blue where they need to be — and HDMI reaches this same mode register for an unrelated purpose (§15.2). The *Parallax Propeller 2 Documentation* describes the converter as intended for baseband video modulation, and usable as a general-purpose RF modulator.

The converter transforms **DAC channels**, not the streamer's pin data. Whatever arrives at DAC3, DAC2 and DAC1 is what it works on — those three are intended to serve as **R, G and B** — while DAC0 is the fourth input and carries control rather than color. Whenever the converter is in use there is a **group delay of five clocks** from DAC-channel input to output.

Five instructions configure it, each writing one parameter register:

Instruction	Parameter	Operand	What it holds
SETCY	CY	D[31:0]	the Y row of the matrix, and a Y offset
SETCI	CI	D[31:0]	the I row, and an I offset
SETCQ	CQ	D[31:0]	the Q row, and a Q offset
SETCFRQ	CFRQ	D[31:0]	the modulation frequency
SETCMOD	CMOD	D[8:0]	every mode selection below, and the digital-video field of §15.2

The matrix

CY, CI and CQ each pack **three coefficients and an offset** into 32 bits: the top three bytes are the coefficients applied to DAC3, DAC2 and DAC1, and the low byte is an offset applied later. The three products are summed and divided by 128.

```
Y[7:0] = (DAC3 × CY[31:24] + DAC2 × CY[23:16] + DAC1 × CY[15:8]) / 128
I[7:0] = (DAC3 × CI[31:24] + DAC2 × CI[23:16] + DAC1 × CI[15:8]) / 128
Q[7:0] = (DAC3 × CQ[31:24] + DAC2 × CQ[23:16] + DAC1 × CQ[15:8]) / 128
```

The divide by 128 is what makes a coefficient readable as 128ths — and **`CMOD[4]` decides how those coefficient bytes are read.** Set, they are **sign-extended**, which is the range a color matrix needs, since Y/I/Q rows carry negative terms. Clear, they are **zero-extended**, and the *Parallax Propeller 2 Documentation* states the consequence directly: with zero-extension, using 128 for a term results in no attenuation of the related DAC term. The same byte `$80` is +128 under one setting and −128 under the other, so a coefficient set is only meaningful alongside the `CMOD[4]` it was computed for.

The modulator

The modulator is what turns the I and Q terms into a color subcarrier. It runs off a phase accumulator named `PHS`, advanced by `CFRQ` rather than by the streamer's NCO frequency word.

```
PHS[31:0] := PHS[31:0] - CFRQ[31:0]
IQ[7:0]   = the Q of (I,Q) after rotation by PHS[31:24], scaled by 1.646
```

Subtracting CFRQ each clock is what produces a **clockwise** angle rotation in the upper bits of PHS; `PHS[31:24]` is then the angle by which the coordinate pair (I, Q) is rotated, and the rotated Q coordinate becomes IQ.

The 1.646 is a rotator artifact you have to budget for. The rotation is performed by a five-stage CORDIC, which scales its result by 1.646. That gain lands on IQ, so the CI and CQ terms must be computed small enough to absorb it — otherwise IQ overflows, and nothing reports that it did.

CAUTION

CFRQ is scaled by 2^{32}; the streamer's NCO is scaled by 2^{31}. The two frequency words look alike and are computed differently, so a value carried from one to the other is wrong by a factor of two.

```
CFRQ      = $1_0000_0000 × modulation_frequency / clock_frequency
NCO §3.2  = $8000_0000   × desired_rate         / clock_frequency
```

For the NTSC color subcarrier of 3.579545 MHz at an 80 MHz clock, the *Parallax Propeller 2 Documentation* works this through to `$0B74_5CFE`. §15.3's `$03AA_5B33` is the same subcarrier at 250 MHz.

Output selection — `CMOD[6:5]`

Four selections decide which computed term reaches each DAC pin:

`CMOD[6:5]`	Mode	DAC3	DAC2	DAC1	DAC0
`%00`	off (bypass)	DAC3	DAC2	DAC1	DAC0
`%01`	VGA (R-G-B) / HDTV (Y-Pb-Pr)	FY (R / Y)	FI (G / Pb)	FQ (B / Pr)	FS (H-Sync)
`%10`	NTSC/PAL Composite + S-Video	FYC (Composite)	FYC (Composite)	FIQ (Chroma)	FYS (Luma)
`%11`	NTSC/PAL Composite	FYC	FYC	FYC	FYC

`%00` is the pass-through selection, and it is the one every DAC example in Chapters 10, 11 and 17 is written for: each channel reaches its pin unmodified. §15.1's VGA program is in that group — it never calls **SETCMOD**, because `X_RFWORD_RGB16` has already put red, green and blue on DAC3, DAC2 and DAC1 and no transformation is wanted. `%01` is for when a transformation *is* wanted on an analog RGB output — component Y-Pb-Pr being the obvious case — and it also puts the converter's own H-sync term on DAC0.

`%10` is S-Video, and it is the only selection that separates luma from chroma: DAC1 carries chroma and DAC0 carries luma, while DAC3 and DAC2 still carry the combined composite signal. One four-channel group therefore feeds an S-Video connector and a composite output at the same time. `%11` is composite alone, on all four channels.

What the low bits of CMOD do

`CMOD[3:0]` control how DAC0 and the offsets enter the output terms:

Bit	Effect
`CMOD[3]`	adds DAC0 into FY — the R / Y term
`CMOD[2]`	adds DAC0 into FI — the G / Pb term
`CMOD[1]`	adds DAC0 into FQ — the B / Pr term
`CMOD[0]`	selects the polarity of FS, the H-sync term

Each output term also takes the **low byte of its own parameter as an offset** — FY adds `CY[7:0]`, FI adds `CI[7:0]`, FQ adds `CQ[7:0]`. That is what the fourth byte of a 32-bit CY/CI/CQ value is for, and it is how a pedestal or black level is set.

In the composite selections the luma term is not a single expression. **`DAC0[1:0]` selects between three luma levels** — sync, blank/burst, and visible — which is how composite sync and the color burst are generated without a separate signal path:

DAC0[1:0]	Luma term FYS
`%1x`	sync level (zero)
`%01`	blank / burst level — taken from `CI[7:0]`
`%00`	visible — `CY[7:0]` plus the matrix Y

The composite output is then that luma term plus the modulated chroma: FYC = FYS + IQ.

HARDWARE

What this guide does not carry. The exact expressions for the seven output terms — FY, FI, FQ, FS, FIQ, FYS and FYC — are stated bit by bit in the COLORSPACE CONVERTER section of the *Parallax Propeller 2 Documentation v35 - Rev B/C*, and this guide does not reproduce them. Everything above is what a streamer program has to choose: which parameter holds what, which selection routes which term to which pin, and which bits gate DAC0 in.

Nor does any source here carry a worked NTSC or PAL coefficient set. The *Parallax Propeller 2 Documentation* gives the mechanism and no coefficients; deriving a set is a video-encoding problem rather than a streamer one, and it depends on the `CMOD[4]` extension mode, the 1.646 rotator gain, and the pedestal the target standard expects.

`CMOD[8:7]` is not part of any of this — it selects digital-video serialization, which §15.2 covers.

15.1 VGA Output

VGA uses analog RGB on DAC channels, with **separate** horizontal and vertical sync. The streamer drives the analog levels: the horizontal-sync level rides the streamer's immediate operand, while vertical sync is a plain pin toggle.

Hardware Requirements:

- Three DAC pins for R, G, B, plus one DAC pin for the horizontal-sync level — each configured per §11.0, with its low two bits selecting the channel it is to carry (§11.2)
- One additional digital pin for vertical sync (toggled directly, not streamed) — a digital pin, so it needs none of that setup
- Resistor DAC network or direct DAC output

The program below assumes those four DAC pins are already configured; it shows the streaming, not the pin setup.

Timing Structure (640×480 @ 60 Hz):

Element	Pixels	Duration @ 25.0 MHz
Visible	640	25.60 µs
Front porch	16	0.64 µs
Sync pulse	96	3.84 µs
Back porch	48	1.92 µs
Total line	**800**	**32.00 µs**

The pixel counts are the VESA standard; the pixel clock is not. VESA specifies 640×480 at 25.175 MHz, which has no jitter-free sysclk a 20 MHz crystal can reach — so this program runs the 25.0 MHz substitute §3.4 works through, exactly ten sysclk cycles per pixel at 250 MHz. That stretches the line from 31.78 µs to 32.00 and the frame from 59.94 Hz to 59.5, inside the tolerance §3.4 quantifies. Its frequency word is the same `$0CCC_CCCD` §15.2's HDMI program uses, and for the same reason: one pixel every ten clocks.

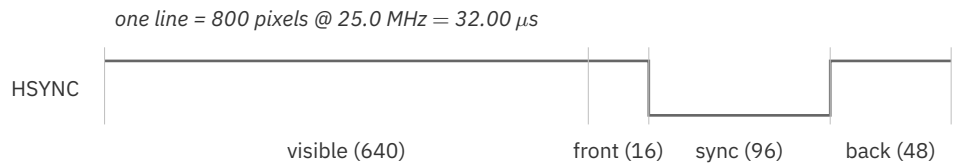


Figure 15.1. VGA 640×480 horizontal line timing

Example:

The non-visible intervals — front porch, sync, back porch, and whole blank lines — stream a **fixed DAC level** through an *immediate* mode, `X_IMM_1X32_4DAC8 | X_DACS_3_2_1_0` (`$7F01_0000`). Its S

operand is the 32-bit level held across the four DAC channels for `D[15:0]` pixels, so the **horizontal-sync** level is simply a different S value during the sync interval. **Vertical sync is not streamed** — it is a separate pin toggled with `DRVNOT` around the vsync lines.

The line and field timings are the full VESA 640×480; the painted height is not. A 640×480 framebuffer at 16 bits per pixel is 600 KB and hub RAM holds 512 KB (§7.1), so the program paints 350 lines and blanks the rest of the field — the same trade §15.2's HDMI program makes, for the same reason. The 525-line field is unchanged, so a monitor still sees standard VGA; it sees black where the framebuffer ran out. The four DAC pins are set up per §11.0 before anything streams: DAC mode carrying this cog's ID, `DIRH`, and a base that is a multiple of 4 so each pin's low two bits land on its own channel.

```
CON   VGA_BASE = 16                     ' four DAC pins, 19..16; a
                                        '  multiple of 4, so their low
                                        '  two bits pick DAC0..DAC3 (11.2)

DAT             ORG
                ' Four DAC pins, one per streamer DAC channel (11.0)
                COGID   cogn                    ' this cog, 0..7
                SETNIB  dacmode, cogn, #2       ' COGID -> M[3:0]
                WRPIN   dacmode, #3<<6 + VGA_BASE
                DIRH    #3<<6 + VGA_BASE        ' pins 19..16 drive

                SETXFRQ pixfreq                 ' 25.0 MHz pixel NCO
                                                ' (2^31-scaled)
                MOV     vsync_pin, #VGA_BASE + 4  ' VSYNC: a separate
                                                ' digital pin
                DRVC    vsync_pin               ' establish initial
                                                ' VSYNC level

vfield          MOV     y, #90                  ' top blanking lines
                CALL    #blank
                RDFAST  ##640*350*2/64, ##framebuffer
                MOV     y, #350                 ' visible lines
line            CALL    #hsync
                XCONT   m_visible, #0           ' 640 RGB pixels
                                                ' (pipeline: Chapter 7)
                DJNZ    y, #line
                MOV     y, #83                  ' bottom blanking lines
                CALL    #blank
                DRVNOT  vsync_pin               ' VSYNC active
                MOV     y, #2                   ' vertical sync (lines)
                CALL    #blank
                DRVNOT  vsync_pin               ' VSYNC inactive
                JMP     #vfield

' One non-visible line: horizontal sync, then a flat blank level
blank           CALL    #hsync
                XCONT   m_blank, #0
          _ret_ DJNZ    y, #blank
```

continues on next page →

↪ continued from previous page

```
' Horizontal sync drives the immediate S operand (the streamed DAC level):
' #0 = blank (0 V); #1 is a PLACEHOLDER sync level — replace #0/#1 with
' your hardware's calibrated blank and sync DAC values.
hsync           XCONT   m_front, #0              ' 16px front porch
                XZERO   m_sync,  #1              ' 96px hsync, phase zero
          _ret_ XCONT   m_back,  #0              ' 48px back porch

' Immediate level mode X_IMM_1X32_4DAC8 ($7001_0000) | X_DACS_3_2_1_0
' ($0F00_0000) = $7F01_0000; D[15:0] = pixel count for the interval.
m_front         LONG    $7F01_0000 + 16
m_sync          LONG    $7F01_0000 + 96
m_back          LONG    $7F01_0000 + 48
' whole line, no visible pixels
m_blank         LONG    $7F01_0000 + 800
' X_RFWORD_RGB16 | X_PINS_ON | X_DACS_3_2_1_0 (route RGB to the DACs)
m_visible       LONG    $BF85_0000 + 640

pixfreq         LONG    $0CCC_CCCD               ' 25.0 MHz @ 250 MHz
dacmode         LONG    P_DAC_124R_3V | P_CHANNEL
cogn            RES     1
vsync_pin       RES     1
y               RES     1

                ORGH    $1000
' 640 x 350 pixels, 16bpp 5:6:5. At 2 bytes/px a full 640 x 480
' frame would be 600 KB and would not fit in hub RAM (7.1)
framebuffer     LONG    0[640*350/2]
```

For a worked reference using this general approach, see Eric R. Smith's *VGA Text routines* (Parallax OBEX #2847).

15.2 HDMI/DVI Output

Digital video is a **streamer** facility rather than a colorspace-converter one, even though it is switched on through the same `CMOD` register. In DVI mode the streamer serializes its internal 32-bit pin output `P[31:0]` into an eight-pin, ten-bit digital-video format: the 32-pin output becomes `$0000_00xx`, and those eight low bits leave as four differential pairs — red, green, blue, and clock.

Hardware Requirements:

- Eight pins in sequence for the four differential pairs
- 1 mA pin drive strength
- System clock = 10 × pixel clock

`CMOD[8:7]` selects the format, and the pair order is reversible. Offsets below are from the eight-pin group's base pin; `P[31:8]` passes through in Normal mode and is forced to `$00_0000` in both DVI modes.

Pin offset	Normal (%0x)	DVI fwd (%10)	DVI rev (%11)
+7	P[7]	RED+	CLK-
+6	P[6]	RED-	CLK+
+5	P[5]	GRN+	BLU-
+4	P[4]	GRN-	BLU+
+3	P[3]	BLU+	GRN-
+2	P[2]	BLU-	GRN+
+1	P[1]	CLK+	RED-
+0	P[0]	CLK-	RED+

The two DVI selections put the same eight signals out in opposite order; the block below selects the forward one.

Sync rides inside the stream, and `P[1]` is what carries it. Eight-bit red, green and blue pixel data are encoded into 10-bit TMDS patterns for transmission, while control data — the horizontal and vertical syncs — is transmitted literally. `P[1]` of the internal pin-output data selects between the two:

P[31:0]	What the three serial channels carry
%RRRRRRRR_GGGGGGGG_BBBBBBBB_xxxxxx0x	each 8-bit channel **gets TMDS-encoded**
%rrrrrrrrrr_gggggggggg_bbbbbbbbbb_1x	each 10-bit channel **is sent literally**

With `P[1]` clear the word holds three 8-bit color channels and the hardware encodes them. With `P[1]` set it holds three ready-made 10-bit patterns and the hardware passes them straight out — which is how a sync interval is expressed as streamed pixel data rather than as a separate pin, the way §15.1's VGA program has to do it.

The system clock is fixed at ten times the pixel rate, because every pixel becomes ten serial bits: 640×480 digital video has a 25 MHz pixel rate, so the P2 runs at 250 MHz. The streamer's NCO is then set to one tenth of the system clock with `$0CCC_CCCC+1`, where the `+1` forces the initial NCO rollover on the tenth clock.

A 640x350 image in a 480-line field

The program below has the shape of §15.1's: the same 800-pixel line, the same 525-line field, the same sync-then-stream loop. What changes is the output stage — eight digital pins rather than four DACs — and the visible height, which is 350 lines because a 640x480 framebuffer at 16 bits per pixel is 600 KB and hub RAM is 512 KB (§7.1). The lines left over are blanked, which is what the top and bottom counts do.

```
CON   HDMI_BASE = 16                    ' must be a multiple of 8

DAT             ORG

                ' Eight pins in sequence, 1 mA drive
                DRVL    #7<<6 + HDMI_BASE         ' pins 16..23
                WRPIN   ##P_HIGH_1MA | P_LOW_1MA, #7<<6 + HDMI_BASE

                SETCMOD #$100                     ' CMOD[8:7] = %10, DVI fwd
                SETXFRQ ##$0CCC_CCCC+1            ' 1/10 sysclk = 25 MHz px
                RDFAST  ##640*350*2/64, ##picture ' wrap on the image

vfield          MOV     hsync0, sync_000          ' VSYNC inactive
                MOV     hsync1, sync_001
                CALLPA  #90, #blank               ' top blanking lines
                MOV     x, #350                   ' visible lines
line            CALL    #hsync
                XCONT   m_visible, #0             ' 640 pixels from the FIFO
                DJNZ    x, #line
                CALLPA  #83, #blank               ' bottom blanking lines
                MOV     hsync0, sync_222          ' VSYNC active
                MOV     hsync1, sync_223
                CALLPA  #2, #blank                ' vertical sync lines
                JMP     #vfield

' One non-visible line: sync, then 640 blanked pixels
blank           CALL    #hsync
                XCONT   m_blank, hsync0
      _ret_     DJNZ    pa, #blank

' Horizontal sync: 16 before, 96 of sync, 48 after
hsync           XCONT   m_front, hsync0
                XZERO   m_sync,  hsync1
      _ret_     XCONT   m_back,  hsync0

' X_IMM_1X32_4DAC8 | X_PINS_ON ($7081_0000); D[15:0] = pixel count
m_front         LONG    $7081_0000 + HDMI_BASE<<17 + 16
m_sync          LONG    $7081_0000 + HDMI_BASE<<17 + 96
m_back          LONG    $7081_0000 + HDMI_BASE<<17 + 48
m_blank         LONG    $7081_0000 + HDMI_BASE<<17 + 640
' X_RFWORD_RGB16 | X_PINS_ON -- pins only, no DAC routing
m_visible       LONG    $B085_0000 + HDMI_BASE<<17 + 640

' Blanking patterns: red, green and blue as ready-made 10-bit
' codes, sent literally because P[1] = 1
sync_000        LONG    %1101010100_1101010100_1101010100_10
sync_001        LONG    %1101010100_1101010100_0010101011_10
sync_222        LONG    %0101010100_0101010100_0101010100_10
sync_223        LONG    %0101010100_0101010100_1010101011_10

x               RES     1
hsync0          RES     1
hsync1          RES     1
```

continues on next page →

↪ continued from previous page

```
                ORGH    $1000
' 640 x 350 pixels, 16bpp 5:6:5. At 2 bytes/px a full 640 x 480
' frame would be 600 KB and would not fit in hub RAM (7.1)
picture         LONG    0[640*350/2]
```

The four sync patterns are the `P[1]` mechanism in use. Each is three 10-bit fields — red, green and blue, in the order the table above gives — with `P[1]` set, so the streamer sends them out literally instead of TMDS-encoding them. `hsync0` carries the pattern for the rest of the line and `hsync1` the one for the sync pulse, and the field loop swaps both when vertical sync begins. The codes themselves are the ones the *Parallax Propeller 2 Documentation*'s HDMI program uses; what a given code means is an HDMI/DVI encoding matter this guide does not cover.

XZERO appears once per line, on the sync pulse, which is §4.7's line-boundary practice: the phase is reset where a glitch would be invisible.

HARDWARE

One register, two facilities. `CMOD[8:7]` serializes the streamer's **pin** output; `CMOD[6:5]` and the bits below it transform the cog's **DAC channels** (§15.0). That is why **SETCMOD** appears in an HDMI program that never uses the converter — and it is worth decoding the constant above: `#$100` is `%1_0000_0000`, so `CMOD[8:7]` = `%10` and `CMOD[6:5]` = `%00`, converter off. The *Parallax Propeller 2 Documentation* documents digital video inside its streamer section and the colorspace converter in a section of its own.

HARDWARE

Blanking intervals are display-limited, not analog-mandated. DVI/HDMI has no analog front/back-porch requirement, so the large blanking intervals inherited from VGA can be trimmed hard — the practical floor is whatever the attached display tolerates. Observed horizontal-blanking floors range from about **16 pixels** on permissive TVs to roughly **68** on older DVI monitors; minimal vertical blanking of about **8 lines** (one sync line plus seven blank) has been driven successfully. Treat these as *observed display limits* to test against your own monitor, not as P2 limits.

CAUTION

Carrying HDMI audio needs more horizontal blanking than video alone. The data-island packets that carry sound need room in each horizontal blanking interval that the tightest video-only timing does not leave — so an audio-carrying design cannot use the tightest blanking. Size the blanking budget from the HDMI data-island specification for your exact mode before committing a timing.

15.3 Composite Video

Composite video uses the colorspace converter to generate NTSC or PAL signals. It is the one output in this chapter that needs the whole of §15.0: the matrix to produce Y, I and Q, the modulator to put I and Q on a color subcarrier, and `DAC0[1:0]` to switch the luma term between sync, burst and visible.

`CMOD[6:5] = %11` puts the combined composite signal on all four DAC channels. **`%10` is the alternative worth knowing about:** it keeps composite on DAC3 and DAC2 while splitting luma and chroma onto DAC0 and DAC1, so the same four channels also drive an S-Video connector.

Pattern — supply `cy_ntsc`, `ci_ntsc` and `cq_ntsc`.

```
' Composite mode (NTSC/PAL) — CMOD[6:5] = %11
SETCMOD #%11 << 5

' NTSC chroma carrier 3.579545 MHz at 250 MHz clock
' CFRQ = $1_0000_0000 × carrier / clkfreq
SETCFRQ ##$03AA_5B33

' Color matrix coefficients
SETCY   ##cy_ntsc
SETCI   ##ci_ntsc
SETCQ   ##cq_ntsc
```

Why the coefficients are left to you. §15.0 gives the shape they have to take — three signed coefficients and an offset in each of CY, CI and CQ, read according to `CMOD[4]`, with the rotator's 1.646 gain budgeted into CI and CQ so IQ cannot overflow. It does not give a worked NTSC or PAL set, and neither does any other source this guide draws on. Deriving one is a video-encoding exercise rather than a streamer one, and a set lifted from a driver built for a different extension mode, pedestal or DAC loading goes wrong in ways that look like a tint error rather than a fault. So this block shows where each parameter goes, not what to put in it.

It also stops short of a picture. Composite still needs a framebuffer, a streamer command and a line loop around this configuration; §15.1 and §15.2 both work that structure through, and only the output stage differs.

CAUTION

Composite is where §3.5's crystal-accuracy warning binds. `CFRQ` is computed from `clkfreq`, so the color subcarrier inherits whatever error the crystal has — and the colorburst is the one video case with only a few ppm of headroom before the hue drifts.

Chapter 16: High-Speed Serial (SPI)

Not every streamer job is video or audio. This chapter shows the streamer as a fast, precise bit pump for serial protocols such as SPI — emitting a stream of bits from memory while a smart pin generates the matching clock. The pairing is the point: the streamer handles the data, the smart pin handles the clock, and both run from the same system clock, so their rates match for transfers far faster than a software bit-bang. Their starting phase is a separate matter, set by your code; §16.1 shows where.

16.1 SPI Output with Streamer

The streamer outputs SPI data while a smart pin generates the clock.

Configuration. Pattern — supply `spi_clk` and `spi_do`.

```
                ' Configure clock pin as transition counter
                WRPIN   ##P_TRANSITION | P_OE, #spi_clk
                WXPIN   ##1, #spi_clk           ' base period in clocks
                                                ' (2 sysclks/clock cycle =
                                                ' one NCO-÷2 data bit)
                DRVL    #spi_clk

                ' Enable the DATA pin too. The streamer supplies this pin's
                ' output state; DIR is still what lets it drive (11.0).
                DRVL    #spi_do

                ' NCO at half clock rate
                SETXFRQ ##$4000_0000
```

Single Byte Transfer. Pattern — supply `spi_do`, `spi_clk` and a `bmode` register; the byte arrives in `pa`.

```
spi_byte        MOV     bmode, ##X_IMM_32X1_1DAC1 | X_PINS_ON | X_ALT_ON
                ADD     bmode, ##spi_do<<17 + 8
                XINIT   bmode, pa               ' Output 8 bits
                WYPIN   #16, #spi_clk           ' 16 clock transitions
          _ret_ WAITXFI
```

Bulk Transfer. Pattern — supply `spi_do`, `spi_clk` and an `rmode` register; `ptra` points at the 256 bytes.

```
spi_block       RDFAST  #0, ptra                ' Point to data
                MOV     rmode, ##X_RFBYTE_1P_1DAC1 | X_PINS_ON | X_ALT_ON
                ADD     rmode, ##spi_do<<17 + 256*8
                XINIT   rmode, #0               ' Stream 256 bytes
```

continues on next page →

↪ continued from previous page

```
              WYPIN   ##256*8*2, #spi_clk     ' Clock transitions
_ret_         WAITXFI
```

CAUTION

The spacing between `XINIT` and `WYPIN` sets where each data bit lands against the clock. The clock starts on its next base-period boundary after `WYPIN`, while the streamer starts as soon as `XINIT` engages. So that spacing picks one of a few phases (one per system clock of the clock's half-period), and exactly one of them loses: the transfer completes, nothing reports an error, and the receiver latches the wrong bits. A spacing that is safe at one clock rate can be the losing one at another, so check the alignment with a logic analyzer at every SCK rate you use.

16.2 Coordinating with WAITXFI

The **WAITXFI** instruction blocks only until the streamer finishes — it has no knowledge of the smart-pin clock. Check the clock's completion separately (e.g. `TESTP` on the clock pin):

Pattern — supply `mode`, `data`, `transitions` and `clk_pin`.

```
              XINIT   mode, data
              WYPIN   transitions, #clk_pin
              WAITXFI                       ' Wait for data
              TESTP   #clk_pin wc           ' Verify clock done
```

CAUTION

The smart pin clock and streamer operate independently. Verify both complete before starting the next transfer.

Chapter 17: Signal Processing

This chapter returns to the DDS and Goertzel capabilities of Chapter 10 and puts them to work — generating waveforms with a function generator's precision, and detecting specific frequencies for tone decoding, distance sensing, and measurement. Where Chapter 10 explained the mechanism, this chapter shows the applications.

17.1 Goertzel Frequency Detection

Goertzel analysis reports how much of one chosen frequency is present in an incoming signal. It is the narrowest measurement the streamer offers, and it is sharp: a 1 MHz detector run against a 1 MHz tone on silicon returned a magnitude of **1,059,000**, while the same detector against the same signal path returned **2,575** at twice the frequency, **286** at half, and **430** with no tone at all — selectivity of roughly **411:1**, **3,700:1**, and a **2,460:1** null.

Application: Ultrasonic distance measurement, DTMF decoding, tone detection

The input is a four-pin block, not a pin

The command's `D[22:19]` field selects a **block of four pins**; the block's base pin is `%pppp` × 4 (documented behaviour — the *Parallax Propeller 2 Documentation v35 - Rev B/C* states the block arithmetic). The block is only half the selection. The **`S` operand chooses what happens to those four pins**:

S field	Purpose
`S[15:12]`	which of the four pins are **summed** — **mandatory**
`S[19:16]`	which of the four are **inverted** (lets a channel be subtracted)
`S[11:0]`	loop size and LUT window

`S[15:12]` = 0 sums nothing, and the analyzer accumulates zero. It is an easy detector to build and a hard one to diagnose: everything else is correct, the command issues, the loop runs, and every magnitude is noise. Supplying `S` is not optional.

Each selected pin contributes ±1 per clock — an input `0` counts as −1 and a `1` as +1. Where two or four channels are summed, the total is always even and is shifted right one bit.

Setup

The ADC pins feeding Goertzel are **raw delta-sigma bitstreams**. Configure them for ADC mode with the smart-pin mode field at `%00000`, and **do not raise DIR** — a smart pin left enabled on these pins produces no accumulation at all. (This is the reverse of the scope-fed ADC modes in §9.2, which require an enabled smart pin. The *Parallax Propeller 2 Documentation*'s own worked program follows the raw form: only the DAC pin gets `DIRH`.)

Gain is a property of the **coupling**, not of this mode. A high-gain constant such as `P_ADC_100X` saturates on a directly-wired signal and reads a constant; it suits a capacitively-coupled touch pad, which is what the demo shipped with the *Parallax Propeller 2 Documentation* uses. A directly-coupled signal wants low gain.

Pattern — supply `adc_pin` and the `sine_table` §10.4 builds.

```
                ' Load sine/cosine table to LUT
                SETQ2   #$200-1
                RDLONG  0, ##sine_table

                ' ADC pin: RAW — ADC mode, smart-pin field %00000, NO DIR
                WRPIN   ##P_ADC_1X, #adc_pin
```

This section builds a **detector** — the streamer's DAC routing stays off, and the only pin configured is the input. The same mode also *generates*, driving a synthesized waveform out of the DACs while it measures; that side is §17.2, and the DAC routing field is what turns it on.

Reading the result: take the difference

`GETXACC` returns the cosine accumulator in `D` and places the sine into the **next instruction's S operand** — which is what the `0-0` placeholder below receives.

Rev C A read clears the accumulators only while a Goertzel command is running. With the streamer idle it returns the running total and clears nothing: a second idle read returns *the same numbers*, and an absolute read after a discrete burst includes every burst before it. That fails invisibly, because the number returned is large, stable and entirely plausible.

So with a discrete `XINIT` / `WAITXFI` / `GETXACC` sequence, read before the command and after it and take the **difference**. Rev C Each burst's last term is held back and added on the first clock of the next Goertzel burst, so for an exact SINC1 sum, deliver it first: run the same command with its count set to 4 and `S[15:12]` = 0, then `WAITXFI`, before each idle read. The `XCONT` loop below reads inside the running command, where the read does clear, so it reads once per command.

Detection loop

Pattern — supply `target_freq`, `cycles`, `threshold`, a `#detected` routine, and the `clkf`, `xfrq`, `cos_acc`, `sin_acc` and `magnitude` registers. §17.2 works the same command word through as a complete program.

```
' Calculate NCO frequency for target (2^31-scaled for the NCO)
                RDLONG  clkf, #$44              ' clkfreq from hub $44
                QFRAC   target_freq, clkf       ' QFRAC = 2^32 × target/clk
                GETQX   xfrq
                SHR     xfrq, #1                ' halve to the NCO's
                                                ' 2^31 scaling
```

continues on next page →

↪ continued from previous page

```
                SETXFRQ xfrq
detect
                ' Run Goertzel analysis
                SETWORD dds_cmd, cycles, #0
                XCONT   dds_cmd, dds_s

                ' One read per command; cos in D, sin into the next S
                GETXACC cos_acc
                MOV     sin_acc, 0-0

                ' Convert to magnitude
                QVECTOR cos_acc, sin_acc
                GETQX   magnitude

                ' Check threshold
                CMP     magnitude, threshold wcz
        if_a    CALL    #detected

                JMP     #detect

' Goertzel, four-pin block 0 (pins 0..3), DAC routing off
dds_cmd         LONG    X_DDS_GOERTZEL_SINC1 | X_DACS_OFF
' sum base pin +0 (S[15:12] = %0001), invert none
' S[11:9] = %000 selects the full 512-long window, %T = 0 (see 10.3)
dds_s           LONG    %0000_0001_000_000000000
```

HARDWARE

SINC2 needs a smaller table. SINC1 accumulates directly and takes the full ±127 waveform amplitude. SINC2 double-integrates for sharper selectivity and overflows on a full-scale table — build it at ±10. Either way the DAC bytes are emitted with the MSB inverted — §10.2's `LUT.BYTE[n] ^ $80` — which puts the waveform's **zero crossing** at `$80` and `$7F`, not its extremes: a ±127 table spans `$01` to `$FF`, and a ±10 table only `$76` to `$8A`.

17.2 DDS Waveform Generation

DDS synthesizes arbitrary waveforms at precise frequencies. It is the other half of the mode §17.1 uses: the same command word, with the DAC routing field turned on and no input pin summed. Where the detector measures one frequency in a signal, the generator emits one — from a table you write, at a rate the NCO sets.

Applications: Function generator, audio synthesis, RF modulation

The output path

The output is a DAC channel, so the pin that carries it needs the setup in §11.0 — DAC mode with this cog's ID, `DIRH`, and low two bits selecting the channel — and the command's `%dddd` field has to route a streamer channel onto it. §17.1's detector sets that field to `X_DACS_OFF`; a generator is what turns it on.

Which channel to route follows from the LUT layout. §10.2 sends LUT byte 0 to X0, byte 1 to X1, and so on, so a single-channel generator routes **X0 to DAC0** with `X_DACS_X_X_X_0` (§11.1) and drives a pin whose low two bits are `%00` (§11.2).

The table

§10.4's build loop already produces exactly the table this needs. There it is called `sine_table`, and its `t.BYTE[0]` — commented as the optional DAC output — is the sample DAC0 emits here. A full-window generator uses all 512 longs; §10.3's smaller loop sizes let several waveforms sit in the LUT at once, each picked by the `%A` bits of its own command.

Amplitude follows §10.5: SINC1 takes the full ±127. Selecting SINC2 instead means building the table at ±10, for the reason given there.

A one-channel function generator

```
CON   DDS_PIN = 8                       ' low bits %00 -> DAC0

DAT             ORG

                ' Output pin: DAC mode, this cog's channels, driving
                COGID   cogn
                SETNIB  dacmode, cogn, #2         ' COGID -> M[3:0]
                WRPIN   dacmode, #DDS_PIN
                DIRH    #DDS_PIN

                ' Waveform table -> LUT, all 512 longs
                SETQ2   #$200-1
                RDLONG  0, ##waveform_table

                ' Output frequency, 2^31-scaled for the NCO
                RDLONG  clkf, #$44                ' clkfreq from hub $44
                QFRAC   out_freq, clkf            ' QFRAC = 2^32 x f/clk
                GETQX   xfrq
                SHR     xfrq, #1                  ' halve to the NCO's
                                                  ' 2^31 scaling
                SETXFRQ xfrq

                ' Run until stopped; DAC0 follows the table
                XINIT   dds_cmd, dds_s
stay            JMP     #stay
```

continues on next page →

↪ continued from previous page

```
' SINC1, nothing summed, X0 -> DAC0 only, perpetual count
dds_cmd         LONG    X_DDS_GOERTZEL_SINC1 | X_DACS_X_X_X_0 + $FFFF
' S[15:12] = 0 sums no pin: this command generates, it does not measure
' S[11:0] = %000_000000000 selects the full 512-long window (10.3)
dds_s           LONG    %0000_0000_000_000000000

out_freq        LONG    1_000                       ' 1 kHz
dacmode         LONG    P_DAC_124R_3V | P_CHANNEL

cogn            RES     1
clkf            RES     1
xfrq            RES     1

                ORGH                                ' hub, not cog RAM
' 512 longs; byte 0 is the sample DAC0 emits. 10.4 builds it
waveform_table  LONG    0[512]
```

s is zero here on purpose. §17.1 makes `S[15:12] = 0` the first thing to check when a detector reads noise, and that is right — for a detector. This command only generates and never reads the accumulators, so summing no pin is the correct setting rather than the classic mistake. A command that does both jobs at once supplies an input block in `S[15:12]` *and* a routing field in `%dddd`.

The count is `$FFFF`, which runs the command perpetually (§4.6) — a function generator that stops after a fixed number of steps is not one. `XSTOP`, or the next command, ends it.

TIP

The table is not restricted to a sine. Square, triangle, or a recorded sample all work the same way: the NCO steps through the window and the routed DAC emits the byte it lands on.

Chapter 18: Integration Patterns

The final chapter collects patterns that cut across everything above: double-buffering so display and rendering never collide, splitting work across multiple cogs, and coordinating the streamer with smart pins. These are the techniques that turn a working streamer demo into a robust system.

18.1 Double Buffering

Use two buffers to allow simultaneous rendering and display:

Pattern — supply `buffer_a`, `buffer_b`, `frame_size`, a `#render_frame` routine, and the `display_buf`, `render_buf` and `temp` registers.

```
                ' Buffer addresses
                MOV     display_buf, ##buffer_a
                MOV     render_buf, ##buffer_b

frame_loop      ' Start displaying current buffer
                RDFAST  ##frame_size/64, display_buf

                ' Render to other buffer while displaying
                CALL    #render_frame

                ' Swap buffers
                MOV     temp, display_buf
                MOV     display_buf, render_buf
                MOV     render_buf, temp

                JMP     #frame_loop
```

18.2 Multi-Cog Video

Complex video systems span multiple cogs:

Cog	Function
0	Main application
1	Horizontal timing, pixel streaming
2	Vertical timing, frame sync
3	Sprite rendering

Synchronization via hub flags. **Pattern** — supply the `line_done_flag` hub long and a `temp` register; the two halves run in different cogs.

```
' Cog 1: Signal line complete
                WRLONG  #1, ##line_done_flag

' Cog 2: Wait for line complete, then clear the flag (handshake)
wait_line       RDLONG  temp, ##line_done_flag wz
        if_z    JMP     #wait_line
                WRLONG  #0, ##line_done_flag    ' clear for next line
```

18.3 Streamer + Smart Pin Coordination

Many applications combine streamer I/O with smart pin timing:

Streamer data with smart pin clock. Pattern — supply `data_mode`, `clocks` and `clk_pin`; §16.1 configures both pins.

```
                XINIT   data_mode, #0           ' Start data output
                WYPIN   clocks, #clk_pin        ' Start clock generation
                WAITXFI                         ' Wait for data complete
```

Smart pin trigger for streamer. Pattern — supply `trigger_pin` and `capture_mode`.

```
wait_trigger    TESTP   #trigger_pin wc         ' wait for the event
        if_nc   JMP     #wait_trigger
                AKPIN   #trigger_pin            ' acknowledge it
                XINIT   capture_mode, #0        ' Start capture
```

Part V: Appendices

The appendices are lookup material: the complete mode-encoding table, the symbol quick reference, the frequency-calculation tables, and a troubleshooting guide. Reach for them once you know which mode you need and want the exact bits.

Appendix A: Complete Mode Encoding Table

Reading the D[19:16] column. It is a template, not a value. Lower-case letters are fields you fill; digits are fixed and must be written as shown:

Letter	Meaning
p	pin-select bit. How many there are depends on the pin count — as the count rises, fewer of these bits select a pin and the freed ones become DAC-configuration bits (§12.2)
a	alternate bit order, D[16] — `0` = bottom-first (default), `1` = top-first (§12.4)
b	LUT base address bits [8:5], for the modes that index the LUT

A row with no letters has no field in D[19:16]: those bits are fixed for that mode and writing anything else into them selects a different mode, silently (§12.0). The DDS/Goertzel rows show a single `p` because their four-pin block selector is `D[22:19]`, so its low bit lands here (§13.4).

Table A.1.

D[31:28]	D[19:16]	Mode	Symbol
`%0000`	`%bbbb`	IMM 32×1 → LUT	`X_IMM_32X1_LUT`
`%0001`	`%bbbb`	IMM 16×2 → LUT	`X_IMM_16X2_LUT`
`%0010`	`%bbbb`	IMM 8×4 → LUT	`X_IMM_8X4_LUT`
`%0011`	`%bbbb`	IMM 4×8 → LUT	`X_IMM_4X8_LUT`
`%0100`	`%pppa`	IMM 32×1 → 1-pin + 1-DAC1	`X_IMM_32X1_1DAC1`
`%0101`	`%pp0a`	IMM 16×2 → 2-pin + 2-DAC1	`X_IMM_16X2_2DAC1`
`%0101`	`%pp1a`	IMM 16×2 → 2-pin + 1-DAC2	`X_IMM_16X2_1DAC2`

Continued on next page

Table A.1. (Continued)

D[31:28]	D[19:16]	Mode	Symbol
%0110	%p00a	IMM 8×4 → 4-pin + 4-DAC1	X_IMM_8X4_4DAC1
%0110	%p01a	IMM 8×4 → 4-pin + 2-DAC2	X_IMM_8X4_2DAC2
%0110	%p10a	IMM 8×4 → 4-pin + 1-DAC4	X_IMM_8X4_1DAC4
%0110	%0110	IMM 4×8 → 8-pin + 4-DAC2	X_IMM_4X8_4DAC2
%0110	%0111	IMM 4×8 → 8-pin + 2-DAC4	X_IMM_4X8_2DAC4
%0110	%1110	IMM 4×8 → 8-pin + 1-DAC8	X_IMM_4X8_1DAC8
%0110	%1111	IMM 2×16 → 16-pin + 4-DAC4	X_IMM_2X16_4DAC4
%0111	%0000	IMM 2×16 → 16-pin + 2-DAC8	X_IMM_2X16_2DAC8
%0111	%0001	IMM 1×32 → 32-pin + 4-DAC8	X_IMM_1X32_4DAC8
%0111	%001a	RFLONG 32×1 → LUT	X_RFLONG_32X1_LUT
%0111	%010a	RFLONG 16×2 → LUT	X_RFLONG_16X2_LUT
%0111	%011a	RFLONG 8×4 → LUT	X_RFLONG_8X4_LUT
%0111	%1000	RFLONG 4×8 → LUT	X_RFLONG_4X8_LUT
%1000	%pppa	RFBYTE → 1-pin + 1-DAC1	X_RFBYTE_1P_1DAC1
%1001	%pp0a	RFBYTE → 2-pin + 2-DAC1	X_RFBYTE_2P_2DAC1
%1001	%pp1a	RFBYTE → 2-pin + 1-DAC2	X_RFBYTE_2P_1DAC2
%1010	%p00a	RFBYTE → 4-pin + 4-DAC1	X_RFBYTE_4P_4DAC1
%1010	%p01a	RFBYTE → 4-pin + 2-DAC2	X_RFBYTE_4P_2DAC2
%1010	%p10a	RFBYTE → 4-pin + 1-DAC4	X_RFBYTE_4P_1DAC4
%1010	%0110	RFBYTE → 8-pin + 4-DAC2	X_RFBYTE_8P_4DAC2
%1010	%0111	RFBYTE → 8-pin + 2-DAC4	X_RFBYTE_8P_2DAC4
%1010	%1110	RFBYTE → 8-pin + 1-DAC8	X_RFBYTE_8P_1DAC8
%1010	%1111	RFWORD → 16-pin + 4-DAC4	X_RFWORD_16P_4DAC4
%1011	%0000	RFWORD → 16-pin + 2-DAC8	X_RFWORD_16P_2DAC8
%1011	%0001	RFLONG → 32-pin + 4-DAC8	X_RFLONG_32P_4DAC8
%1011	%0010	RFBYTE LUMA8	X_RFBYTE_LUMA8
%1011	%0011	RFBYTE RGBI8	X_RFBYTE_RGBI8
%1011	%0100	RFBYTE RGB8	X_RFBYTE_RGB8
%1011	%0101	RFWORD RGB16	X_RFWORD_RGB16
%1011	%0110	RFLONG RGB24	X_RFLONG_RGB24
%1100	%pppa	1-pin + 1-DAC1 → WFBYTE	X_1P_1DAC1_WFBYTE

Continued on next page

Table A.1. (Continued)

D[31:28]	D[19:16]	Mode	Symbol
%1101	%pp0a	2-pin + 2-DAC1 → WFBYTE	X_2P_2DAC1_WFBYTE
%1101	%pp1a	2-pin + 1-DAC2 → WFBYTE	X_2P_1DAC2_WFBYTE
%1110	%p00a	4-pin + 4-DAC1 → WFBYTE	X_4P_4DAC1_WFBYTE
%1110	%p01a	4-pin + 2-DAC2 → WFBYTE	X_4P_2DAC2_WFBYTE
%1110	%p10a	4-pin + 1-DAC4 → WFBYTE	X_4P_1DAC4_WFBYTE
%1110	%0110	8-pin + 4-DAC2 → WFBYTE	X_8P_4DAC2_WFBYTE
%1110	%0111	8-pin + 2-DAC4 → WFBYTE	X_8P_2DAC4_WFBYTE
%1110	%1110	8-pin + 1-DAC8 → WFBYTE	X_8P_1DAC8_WFBYTE
%1110	%1111	16-pin + 4-DAC4 → WFWORD	X_16P_4DAC4_WFWORD
%1111	%0000	16-pin + 2-DAC8 → WFWORD	X_16P_2DAC8_WFWORD
%1111	%0001	32-pin + 4-DAC8 → WFLONG	X_32P_4DAC8_WFLONG
%1111	%0010	1 ADC → WFBYTE	X_1ADC8_0P_1DAC8_WFBYTE
%1111	%0011	1 ADC + 8-pin → WFWORD	X_1ADC8_8P_2DAC8_WFWORD
%1111	%0100	2 ADC → WFWORD	X_2ADC8_0P_2DAC8_WFWORD
%1111	%0101	2 ADC + 16-pin → WFLONG	X_2ADC8_16P_4DAC8_WFLONG
%1111	%0110	4 ADC → WFLONG	X_4ADC8_0P_4DAC8_WFLONG
%1111	%p111	DDS/Goertzel SINC1	X_DDS_GOERTZEL_SINC1
%1111	%p111 (D[23]=1)	DDS/Goertzel SINC2	X_DDS_GOERTZEL_SINC2

Appendix B: Symbol Quick Reference

Mode Symbols

```
X_IMM_32X1_LUT          X_IMM_16X2_LUT          X_IMM_8X4_LUT
X_IMM_4X8_LUT           X_IMM_32X1_1DAC1        X_IMM_16X2_2DAC1
X_IMM_16X2_1DAC2        X_IMM_8X4_4DAC1         X_IMM_8X4_2DAC2
X_IMM_8X4_1DAC4         X_IMM_4X8_4DAC2         X_IMM_4X8_2DAC4
X_IMM_4X8_1DAC8         X_IMM_2X16_4DAC4        X_IMM_2X16_2DAC8
X_IMM_1X32_4DAC8        X_RFLONG_32X1_LUT       X_RFLONG_16X2_LUT
X_RFLONG_8X4_LUT        X_RFLONG_4X8_LUT        X_RFBYTE_1P_1DAC1
X_RFBYTE_2P_2DAC1       X_RFBYTE_2P_1DAC2       X_RFBYTE_4P_4DAC1
X_RFBYTE_4P_2DAC2       X_RFBYTE_4P_1DAC4       X_RFBYTE_8P_4DAC2
X_RFBYTE_8P_2DAC4       X_RFBYTE_8P_1DAC8       X_RFWORD_16P_4DAC4
X_RFWORD_16P_2DAC8      X_RFLONG_32P_4DAC8      X_RFBYTE_LUMA8
X_RFBYTE_RGBI8          X_RFBYTE_RGB8           X_RFWORD_RGB16
X_RFLONG_RGB24          X_1P_1DAC1_WFBYTE       X_2P_2DAC1_WFBYTE
X_2P_1DAC2_WFBYTE       X_4P_4DAC1_WFBYTE       X_4P_2DAC2_WFBYTE
X_4P_1DAC4_WFBYTE       X_8P_4DAC2_WFBYTE       X_8P_2DAC4_WFBYTE
X_8P_1DAC8_WFBYTE       X_16P_4DAC4_WFWORD      X_16P_2DAC8_WFWORD
X_32P_4DAC8_WFLONG      X_1ADC8_0P_1DAC8_WFBYTE X_1ADC8_8P_2DAC8_WFWORD
X_2ADC8_0P_2DAC8_WFWORD X_2ADC8_16P_4DAC8_WFLONG X_4ADC8_0P_4DAC8_WFLONG
X_DDS_GOERTZEL_SINC1    X_DDS_GOERTZEL_SINC2
```

Control Symbols

```
X_PINS_OFF    X_PINS_ON     X_WRITE_OFF   X_WRITE_ON
X_ALT_OFF     X_ALT_ON
```

DAC Symbols

```
X_DACS_OFF      X_DACS_0_0_0_0    X_DACS_X_X_0_0    X_DACS_0_0_X_X
X_DACS_X_X_X_0  X_DACS_X_X_0_X    X_DACS_X_0_X_X    X_DACS_0_X_X_X
X_DACS_0N0_0N0  X_DACS_X_X_0N0    X_DACS_0N0_X_X    X_DACS_1_0_1_0
X_DACS_X_X_1_0  X_DACS_1_0_X_X    X_DACS_1N1_0N0    X_DACS_3_2_1_0
```

Appendix C: Frequency Calculation Tables

NCO Frequency Values

Formula: `frequency = $8000_0000 * (rate / clock)`

Rate Ratio	Value	Notes
1:1	$8000_0000	Every clock
1:2	$4000_0000	Half clock
1:3	$2AAA_AAAB	Add 1 for fractional
1:4	$2000_0000	Quarter clock
1:5	$1999_999A	Add 1
1:8	$1000_0000	Eighth clock
1:10	$0CCC_CCCD	Tenth clock
1:16	$0800_0000	Sixteenth clock

Common Video Pixel Rates

Resolution	Pixel Rate	At 250 MHz	At 300 MHz	At 320 MHz
640×480	25.175 MHz	$0CE3_BCD4	$0ABD_C806	$0A11_EB86
640×480	25.000 MHz	$0CCC_CCCD	$0AAA_AAAB	$0A00_0000
720×480	27.000 MHz	$0DD2_F1AA	$0B85_1EB9	$0ACC_CCCD
800×600	40.000 MHz	$147A_E148	$1111_1112	$1000_0000
1024×768	65.000 MHz	$2147_AE15	$1BBB_BBBC	$1A00_0000
1280×720	74.250 MHz	$2604_1894	$1FAE_147B	$1DB3_3334

Values are `$8000_0000 * pixel_rate / clock_frequency`, **truncated, then incremented by 1 whenever that division leaves a remainder** — the increment is required, not a refinement, and it is not the same as rounding to nearest: a remainder well below half still increments. Only three entries in this table divide exactly and take no increment — 25.000, 40.000 and 65.000 MHz, each at 320 MHz. Every other value here carries the +1. Two rates are listed for 640×480: 25.175 MHz is the VESA figure, and 25.000 MHz is the substitute §3.4 recommends on a 20 MHz crystal — ten cycles per pixel at 250 MHz, and what §15.1 runs.

Appendix D: Troubleshooting Guide

Symptom: No Output on Pins

Check:

1. D[23] = 1 (`X_PINS_ON` included in mode)
2. Pin group %ppp selects correct pins
3. Sub-pin selection matches target pins
4. Pins configured as outputs (DRVH/DRVL as needed)

Symptom: Corrupted Data from RDFAST

Check:

1. **RDFAST** executed before streamer command
2. Buffer start address long-aligned (4-byte, ends in `%00`) for wrap mode
3. Buffer size matches FIFO configuration
4. No other code reading from FIFO simultaneously

Symptom: Streamer Stops Unexpectedly

Check:

1. Count field D[15:0] not zero
2. Command buffer has pending command (**XCONT**/**XZERO** issued)
3. No **XINIT #0,#0** executed

Symptom: Phase Drift in Video

Check:

1. Use **XZERO** at line boundaries
2. NCO frequency matches pixel rate exactly
3. Total pixels per line equals timing specification

Symptom: DAC Output Incorrect

Check:

1. %dddd field selects correct routing
2. DAC channel matches pin LSBs
3. Pin configured for DAC mode

Symptom: Goertzel Results Invalid

Check:

1. **`S[15:12]`, the summed-pins field, is not zero.** Zero sums nothing, so the accumulators never move and every magnitude reads as noise, with everything else correct and the loop running (§17.1)
2. **The input pin is RAW — do not enable it.** Goertzel reads a raw delta-sigma bitstream: **WRPIN** an ADC gain constant with the smart-pin mode field at `%00000`, and **leave DIR low**. A smart pin left enabled there accumulates nothing at all. That is the reverse of §9.2's scope-fed ADC modes, which read a smart pin's result and *do* require `DIRH` — so a pin configured the §9.2 way produces exactly this symptom (§17.1)
3. **Gain matches the coupling.** A high-gain constant such as `P_ADC_100X` saturates on a directly-wired signal and reads a constant; it suits a capacitively-coupled touch pad. A directly-coupled signal wants low gain (§17.1)
4. LUT contains signed sine/cosine values
5. Sample count adequate for frequency resolution
6. SINC2 amplitude reduced to ±10 to prevent overflow
7. **SINC2 only:** iteration count per Goertzel cycle is constant — periodic glitches mean a non-power-of-two rate; run at a power-of-two-relationship clock (e.g. 256 MHz for a 1 MHz target) or switch to SINC1 (§10.5)
8. **You compiled with `-d`.** Accumulators reading in the millions where you expect hundreds are the debug interrupt, not your signal — see §14.5

Symptom: Measurements Change When You Add DEBUG

Check:

1. `DEBUG_COGS` is limiting DEBUG to the cogs you actually watch, not the default all-eight (§14.5)
2. The streaming cog is excluded from that mask
3. The measurement was re-taken *after* narrowing the mask — a run made under the default mask is not a baseline

Index

A

- ADC sampling modes: Chapter 9
- Alignment (8-pin window base): 12.0, 12.2
- Alternate bit order: 12.4
- Architecture: Chapter 2

B

- Block diagram: 2.1

C

- CFRQ parameter: 15.0
- Clock accuracy: 3.5
- CMOD register: 15.0, 15.2
- COGID (in a pin's mode word): 11.0
- Color matrix (Y/I/Q): 15.0
- Colorspace converter: 15.0, 15.3
- Command structure: Chapter 4
- Composite video: 15.0, 15.3
- Count field: 4.6

D

- DAC channels: Chapter 11
- DAC mode configuration: 11.0
- DAC pin mapping: 11.2
- DAC routing table: 11.1
- DAC symbols: 13.3
- DDS mode: Chapter 10
- DDS waveform generation: 17.2
- DEBUG interrupt: 14.5
- DEBUG_COGS: 14.5
- Debugging streamer code: 14.5
- DIRH (pin drive enable): 11.0
- Double buffering: 18.1
- DVI forward/reverse: 15.2

E

- Enable control: 12.3
- Events: Chapter 14

F

- Frequency calculation: 3.2, 3.4, Appendix C
- Function generator: 17.2

G

- GETXACC: 4.7, 10.6
- Goertzel frequency detection: 17.1
- Goertzel mode: Chapter 10

H

- HDMI output: 15.2
- Hub FIFO: 6.1

I

- Immediate modes: Chapter 5

J

- Jitter (per-pixel): 3.4, 3.5

L

- Loop size (DDS/Goertzel): 10.3
- Luma/chroma separation: 15.0, 15.3
- LUT setup: 5.1, 10.4
- LUT window: 10.3

M

- Mode encoding table: Appendix A
- Mode field: 4.2
- Mode symbols: 13.1
- Modulation (phase offset): 10.3
- Modulator (colorspace): 15.0
- Multi-cog: 18.2

N

- NCO: Chapter 3

O

- Oscillator (TCXO): 3.5

P

- `P[1]` (literal-transmit bit): 15.2
- `P_CHANNEL`: 11.0, 13.4
- Phase offset (`%T`): 10.3
- Pin field (positional, mode-specific): 12.0
- Pin group selection: 12.1
- Pin selection: Chapter 12
- `pin<<17` idiom: 12.0, 13.4
- Pixel rate: 3.4

R

- RDFAST modes: Chapter 6
- RGB modes: Chapter 7

S

- S-Video: 15.0, 15.3
- `S[11:0]` (loop size and LUT window): 10.3
- SETCMOD: 15.0, 15.2
- SETCY / SETCI / SETCQ / SETCFRQ: 15.0
- SETDACS: 11.0, 11.1
- SETSCP: 9.1
- SETXFRQ: 3.3, 4.7
- Signal processing: Chapter 17
- SINC1/SINC2: 10.5
- Smart pin coordination: 18.3
- SPI: Chapter 16
- Sub-pin selection: 12.2
- Symbol composition: 13.4
- Symbols quick reference: Appendix B

T

- TCXO: 3.5
- TMDS encoding: 15.2

- Troubleshooting: Appendix D

V

- VGA output: 15.1
- Video output: Chapter 15

W

- WAITXFI: 14.2
- WRFAST modes: Chapter 8
- WRPIN (pin mode configuration): 11.0

X

- XCONT: 4.7
- XINIT: 4.7
- XSTOP: 4.7
- XZERO: 4.7